

AliveCorKitLite Framework (1.6.10)

Revision History.....	4
0.1.....	4
0.1.1.....	4
0.2.....	4
0.3.....	4
0.4.....	4
0.4.1.....	5
0.5.0.....	5
0.5.3.....	5
0.5.4.....	6
0.5.5.....	6
0.7.....	6
0.7.1.....	7
0.7.2.....	7
0.7.3.....	7
0.7.4.....	7
0.7.5.....	8
1.0.0.....	8
1.0.3.....	8
1.0.4.....	8
1.0.5.....	8
1.0.6.....	9
1.0.7.....	9
1.1.0.....	9
1.1.1.....	9
1.1.2.....	9
1.1.3.....	9
1.2.0.....	10
1.2.1.....	10
1.3.0.....	10
1.4.0.....	10
1.5.0.....	10
1.5.1.....	10
1.6.0.....	10
1.6.2.....	11
1.6.3.....	11

1.6.4.....	11
1.6.5.....	11
1.6.10.....	11
Overview.....	12
Configuring Project.....	14
Create new Xcode Project.....	14
Add dependencies.....	14
Permissions.....	15
Add AliveCorKitLite Framework.....	15
Add AliveCorKitAssets bundle.....	16
Verify tags.....	16
Build Settings.....	17
Localization.....	17
AliveCorKitLite.....	19
Initializing AliveCorKitLite.....	20
Request SDK authorization token.....	20
Authorize SDK.....	21
Start Recording.....	22
Handling ECG Recording.....	22
Display Legal Information Screen of AliveCorKitLite framework.....	23
Customize Language in AliveCorKitLite framework.....	23
Header & Footer.....	24
Navigation Bar.....	24
Custom Preview Screen.....	25
Displaying Custom Records.....	25
Customizing HUD links.....	27
PDF.....	28
Built-in PDF generation flow.....	28
PDF generation.....	28
PDF Preview.....	29
Classes.....	30
ACKManager.....	30
ACKKardiaAI.....	32
ACEcgRecordingConfig.....	34
ACKConfiguration.....	40
ACKDevice.....	41
ACEcgEvaluation.....	43
ACKDeviceType.....	47
ACKLeadsConfig.....	48

ACKFilterType.....	48
ACKAlgorithmDetermination.....	48
ACEcgRecord.....	49
ACKErrorCode.....	53
ACEcgMonitorViewController.....	55
ACEcgMonitorDelegate.....	57
ACEcgPreviewViewController.....	64
ACEcgPreviewDelegate.....	65
ACKAlgorithmDeterminationsViewController.....	68
ACEcgFileView.....	69
ACKScrollView.....	72
ACKRecordingResultTraceView.....	73
ACKRecordingResultTraceViewDelegate.....	74
ACKPDFInteractionController.....	75
ACKPDFInteractionControllerDelegate.....	77
ACKPDPPreviewController.....	79
ACKPDFConfig.....	80
ACKPDFDisplaySettings.....	81
ACKPDFMetadata.....	82
ACKPDFReport.....	83
ACKPDFErrorCode.....	85
ACKWebLinks.....	86
ACKBluetoothPairingController.....	87
ACKBluetoothPairingControllerDelegate.....	88
ACKStreamingFilter.....	90

Revision History

0.1

Release in December, 2019

Pre-release 0.1 version of SDK

0.1.1

Release in January, 2020

Changes:

- Supports Kardia AI 1.2.1

0.2

Release in January, 2020

Changes:

- ACKEcgRecord's isInverted property moved to the ACKEcgEvaluation
- The ACKEcgMonitorConfig class renamed to the ACKEcgRecordingConfig
- kaiResult type changed from the string type to the ACKAlgorithmDetermination
- duration property added to the ACKEcgRecord
- ACKEcgRecord's localId property renamed to uuid
- originalPath, enhancedPath removed from the ACKEcgRecord init methods
- dirPath property added to the ACKEcgRecord, ACKManager
- ACKEcgRecordingConfig information added to the ACKEcgRecord.
- ACKEcgFileView added to public access

0.3

Released on April 9, 2020

Changes:

- ACKScrollView added to public access
- ACKRecordingResultTraceView added to public access
- Added support of native iOS layouts(constraints, xib) in header & footer customization.
- Added customization samples in the AliveCorKitExample app.
- Added recording confirmation screen that could be enabled/disabled from ACKConfiguration instance by setting showRecordingConfirmation property

0.4

Released on April 26, 2020

Changes:

- Added the [ACKWebLinks](#) class that allows customizing help links that are used on the

HUD views.

- Added the [ACKPDFPreviewController](#) class that allows rendering PDF reports.
- Added the [ACKPDFMetadata](#) class that allows adding patients data to the PDF report.
- Added the [ACKPDFReport](#) class that allows to export ECG recording to the PDF file.
- Added the [ACKPDFConfig](#) class that defines which ECG and MedatDate should be used for the PDF report.
- Added the [ACKPDFInteractionController](#) class that controls the user flow of the PDF creation.

0.4.1

Released on May 3, 2020

Changes:

- Adding invert ecg example to the example app
- Adding convenience methods to the [ACKEcgREcord](#), [ACKEcgFileView](#)
- Exposing KAI evaluation description in [ACKEcgEvaluation](#)
- Adding support of th [ACKDeviceTypeSakuraOne](#)
- Removing [ACKMainsFrequencyUnknown](#)

0.5.0

Released on May 28, 2020

Changes:

- Adding localization support, check localization for more details.
- Renaming resetDuration to [minDuration](#) in the [ACKEcgRecordingConfig](#).
- Sharing public access to BP pairing by using [ACKBluetoothPairingController](#).(documentation is not ready, please refer to the header files).

0.5.3

Released on June 19, 2020

Changes:

- Removing leadsConfig property from [ACKDevice](#).
- Fixing bug with rendering original ECG records.
- Fixing device not found hud (open example.application, start recording, don't touch electrodes for a while, device not found HUD should be displayed).
- Adding "unlock recording" experience for devices other than [ACKDeviceTypeTriangle](#). Check the hideTrace property of the [ACKEcgRecordingConfig](#) class for more details.
- Sharing access to all supported devices.
- Fixing auto inversion.
- Sharing access to the [ACKAlgorithmDeterminationsViewController](#)
- Adding swedish language support.
- Adding support of the legacy evaluation by sharing access to the method [ecgAlgorithmDeterminationForAfibDetected](#) in [ACKEcgEvaluation](#).
- Adding init method with error handling to the [ACKEcgMonitorViewController](#)
- Updating convenience methods of the [ACKEcgRecordingConfig](#).

- Introducing new error `ACKErrorCodeInvalidRecordingConfig`.
- Fixing `PDFMetaData` (sharing name/last name)
- Showing '-- bpm' on the preview screen if ECG recording is too short.

0.5.4

Changes:

- PDF issue with 50/60 Hz mains filter.
- Date of recording printed on the PDF report changes if the timezone changes, while the date shown in the Diary, ECG history, and ECG review remains correct.
- Incorrect message text in 'Recordings Unreadable' prompt.
- After interference, Recording continues past the 30second countdown, showing tick/completed, but never displaying the result screen.
- EKG 'Original filter' is not handled properly in real time recording (for kardia mobile).
- 6L recording should not be auto-inverted.

0.5.5

Changes:

- `appName` property added to the `ACKManager`. Use it to define which app name would be displayed in the warnings/messages. If `appName` is not defined, SDK will try to use name defined in the `CFBundleDisplayName` property
- `analysisDescription` method added to the `ACKEcgEvaluation`
- `disclaimer` method added to the `ACKEcgEvaluation`
- `additionalInformation` method added to the `ACKEcgEvaluation`
- Updated `ACKRecordingResultTraceViewDelegate`. The client app could update height of the `ACKRecordingTraceView` by overriding delegate method `recordingResultTraceViewHeightForLeadsConfig` or by using standard iOS constraints without extra delegate call.
- `stopMonitor` added to the `ACKEcgMonitorViewController`. The client app would need to call this method if creates custom close/back buttons.
- Removed tensorflow library, reduced binary file size.
- `logoName` added to the `ACKPDFConfig`

0.7

Released on Jul 9, 2020

Changes:

- `deviceDisplayName` property added to the `ACKEcgRecordingConfig`. Define this property to customize device name (works only for Omron's devices)
- For the too_short KAI results, `ACKEcgEvaluation` would have 0 value for the `averageHeartRate`
- PDF reports show a disclaimer for the ecg recordings > 30 seconds.
- Copyright of the PDF report is synced with the Android app.

0.7.1

Released on Jul 29, 2020

Changes:

- Updated description for the averageHeartRate, added upper/lower limits information
- Added [didEncounterAudioError](#) delegate method in the [ACKEcgMonitorDelegate](#) that allows override behavior for the audio errors.
- Added ACKPDFErrorCode with the list of errors that could be encountered during report creation.
- Added [evaluateATCSamples](#) method in the [ACKKardiaAI](#) to allow frequency resampling for the ATC ecg recordings.
- Added error support in the [ACKPDFInteractionController](#) to allow monitoring of PDF related errors.
- Added error support in the [ACKPDFReport](#) to allow monitoring of PDF related errors.
- Internal defect fixes.

0.7.2

Released on Aug 4, 2020

Changes:

- Fixed crash when the user moves to the background mode with the switching off the phone and back.
- Added [shouldCaptureAudio](#) property which indicates whether the application should record audio notes or not.
- Added audio notes to the recording [audioNotesPath](#) (ACKEcgRecord)
- Added [didInvertRecord](#) method to the [ACKEcgPreviewDelegate](#). which informs the delegate that invert button was pressed and returns the recent state of the ACKEcgRecord

0.7.3

Released on Aug 21, 2020

Changes:

- Japanese localization fixes.
- Adding additional userinfo in the audio errors.
- Adding [maximumDisplayMode](#) property description in the documentation.

0.7.4

Released on August 31, 2020

Changes:

- Adding da, pl localizations.

- Localization fixes.
- Adding [instructionsURL](#) property to the [ACKEcgRecordingConfig](#). The animation would be shown on the ECG recording screen (ACKEcgMonitorController). Would work only with [ACKDeviceTypeSakuraOne](#).
- Adding [instructionsMessage](#) property to the [ACKEcgRecordingConfig](#). If property is not defined, the default instructions message would be used. Would work only with [ACKDeviceTypeSakuraOne](#).

0.7.5

Released on September 9, 2020

Changes:

- Localization fixes.
- Crash fix for the scenario when ACKPDFReport is used with the wrong atc file path.

1.0.0

Released on October 19, 2020

Changes:

- SDK provisioning & reporting.

1.0.3

Released on December 9, 2020

Changes:

- AppStore upload troubleshooting.
- Defect fixes.

1.0.4

Released on December 11, 2020

Changes:

- Fixing crash when configuration is missing.

1.0.5

Released on January 2, 2021

Changes:

- Fixing a naming conflict of the webex sdk and AliveCorKitLite sdk.

1.0.6

Released on March 5, 2021

Changes:

- Bug fixes.

1.0.7

Released on April 1, 2021

Changes:

- Enabling support of the XCode 12.4.
- Enabling simulator support.

1.1.0

Released on May 19, 2021

Changes:

- Added ACKStreamingFilter.
- Bug fixes.

1.1.1

Released on May 28, 2021

Changes:

- Bug fixes.
- Updating documentation for the ACKManger's [checkStatusWithListener](#).
- Updating documentation for the ACKManger's [initWithApiKey](#).
- Updating documentation for the [ACKStreamingFilter](#).
- Fixing a few bugs in the sample app.

1.1.2

Released on Aug 6, 2021

Changes:

- Added `didReceiveEcgFrame` in the `ACKEcgMonitorController`.

1.1.3

Released on Dec 11, 2021

Changes:

- Bug fixes.

1.2.0

Released on Apr 3, 2022

Changes:

- Added didReceiveHeartRate in the ACKEcgMonitorController.

1.2.1

Released on June 22, 2022

Changes:

- Added forceInvert property to the ACKPDFConfig.

1.3.0

Released on Nov 7, 2022

Changes:

- Bump version number to match the version number of the Android SDK

1.4.0

Released on Mar 7, 2023

Changes:

- Add KardiaMobile Card support
- Bug fix

1.5.0

Released on Jul 14, 2023

Changes:

- Replace FAT by .xcframework.
- Support KAI v2 for eligible customers.
- Extending supported languages.
- Bug fix.

1.5.1

Released on Jan 3, 2024

Changes:

- Included Privacy manifest file.
- Bug fix.

1.6.0

Released on May 3, 2024

Changes:

- Bug fix.

1.6.2

Released in Sep 2024

Changes:

- Added a new method, **[ACKEcgMonitorDelegate ecgMonitorViewController:didReceiveEcgFrame:]**, in **ACKEcgMonitorDelegate** for exposing the real-time ECG data series.
- Added a new variable, **defaultLanguageType**, to **ACKManager** for overriding the device language setting.
- Added a new variable, **languageType**, to **ACKPDFDisplayingSettings** to override the device language setting for PDF files.
- Added **ACKAboutViewController** for presenting AliveCorKitLite legal information.

1.6.3

Release in Oct 2024

Changes:

- Update KardiaAI to v2.0.8.

1.6.4

Release in Feb 2025

Changes:

- Update KardiaAI to v2.0.9.
- Bug fix.

1.6.5

Release in May 2025

Changes:

- Add support for Serbian and Hebrew languages.
- Bug fix.

1.6.10

Release in November 2025

Changes:

- Added support for **Greek, Spanish (U.S.), Tamil, and Malay (Singapore)** languages.

- Added a new variable to display **additional patient information** below the patient's name on the **PDF cover page**.
- Added a new method to **ACKEcgMonitorDelegate** to receive **battery level callbacks** during recording.

Overview

AliveCor SDK paired with Kardia 6L captures a medical grade 30 seconds accurate electrocardiogram(ECG or EKG) recordings and provides clinically-validated result run on F

DA-cleared Kardia-AI algorithm.

SDK provides a drop-in UI which is a complete, ready-made recording UI that offers a quick and easy way to start recording EKG and show FDA-cleared result to customers.

SDK distributed as AliveCorKitLite framework.

Minimum Deployments: **iOS 15.0**

Minimum Xcode Version: **Xcode 16.2**

Swift Language Version: **Swift 5**

AliveCorKitLite - includes a set of features that allows the consumer to record ECG(EKG), to review ECG results, to review ECG evaluation made by Kardia AI.

AliveCorKit - includes **AliveCorKitLite** and integration with **Kardia PRO** back end.



Configuring Project

Create new Xcode Project

In the Xcode application select **File > New > Project** and create a new Application. Select the desired type of Application under iOS and create the Xcode Project.

Add dependencies

Add these frameworks under **Target > General > Linked Frameworks and Libraries**:

CoreBluetooth.framework - is required for Bluetooth connectivity with AliveCor Connected devices: Triangle. (Add this framework only if your app is going to use Triangle or other device that is using Bluetooth)

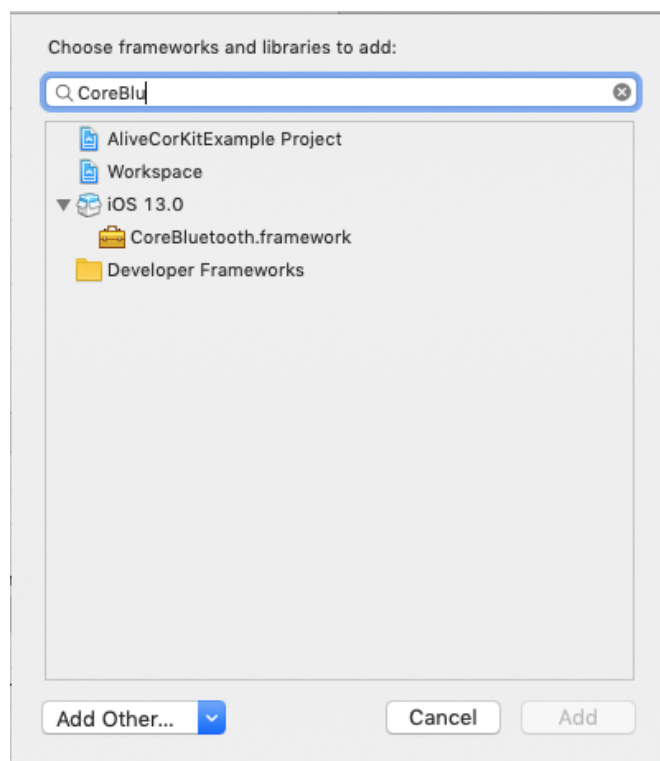
AVFoundation - is required for Audio connectivity with AliveCor devices: Kardia Mobile, Kardia Band. (Add this framework only if your app is going to use KardiaMobile)

UIKit.framework - is required by UI components.

CoreGraphics - is required by UI components.

Foundation.framework

CoreData.framework



Permissions

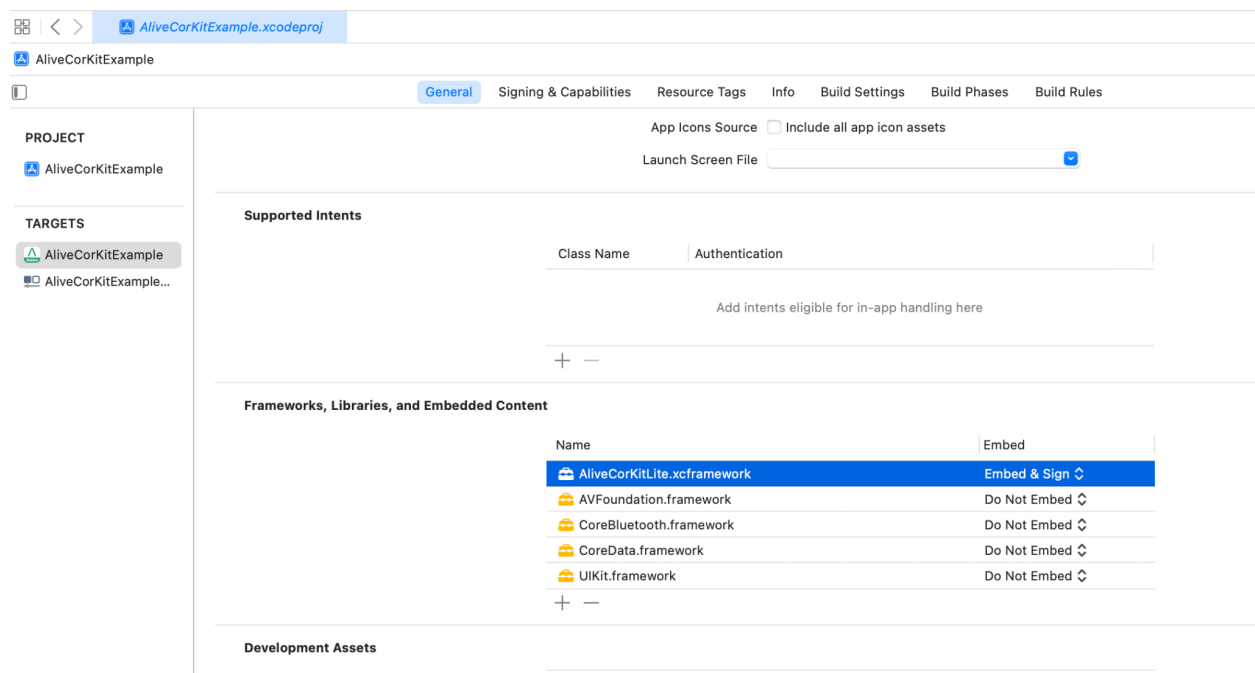
To support a Kardia Mobile device, the app's Info.plist must contain an **NSMicrophoneUsageDescription** key with a string value explaining to the user that the application requires audio permission to be able to communicate with Kardia Mobile to record ECGs.

Add AliveCorKitLite Framework

AliveCorKitLite.xcframework is required to connect to AliveCor Devices. Right click project name and choose “Add Files to...” Option to add this framework to the Xcode project.

Please select “Copy items if needed” when adding the framework to bundle the framework in the Partner application. Once completed, confirm that the target has the framework included.

Make sure that framework is included with ‘Embed & sign’ configuration.



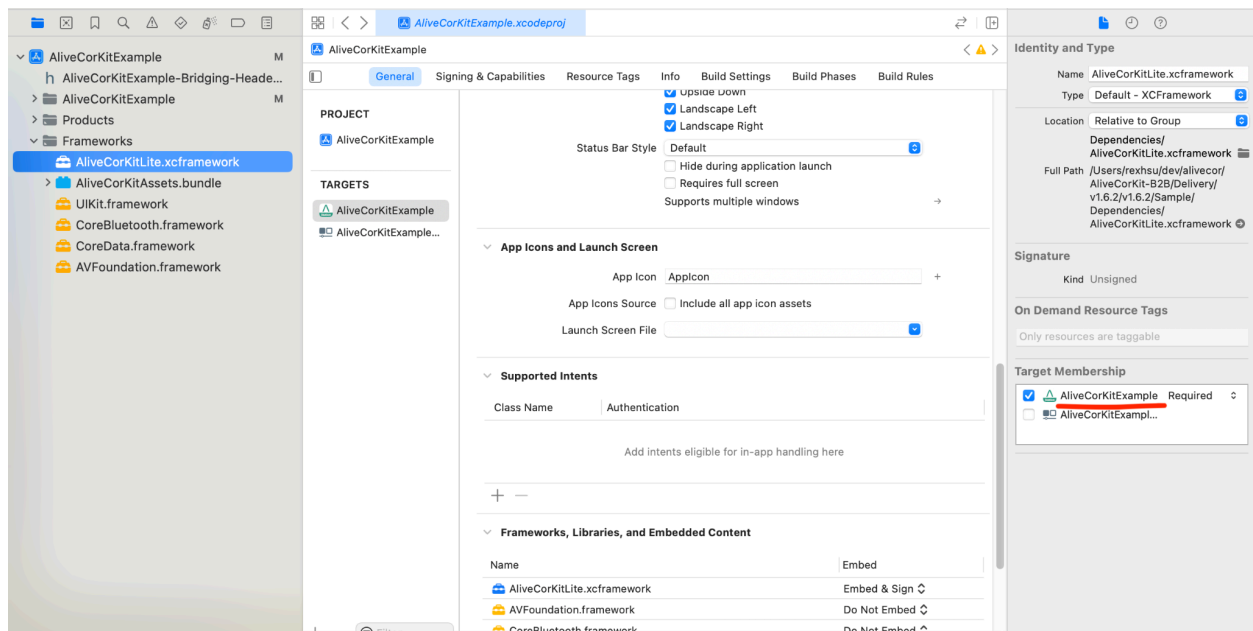
Add AliveCorKitAssets bundle

AliveCorAssets.bundle is required to get the image and thumbnails that are used in **AliveCorKitLite**. Right click the project name and choose “**Add Files to...**” option to add this asset bundle to Xcode project.

Please select “Copy items if needed” when adding the asset bundle to bundle the same in the Partner application. Once completed, confirm that the target has the bundle included.

Verify tags

Verify that the **AliveCorKitLite.xcframework**, **AliveCorKitAssets.bundle** target Membership checkbox is set. Also verify that your Deployment Target is greater than or equal to the deployment target of the library.



Build Settings

iOS deployment target: **15.0**

Targeted device families: **iPhone/iPad**

XCode **16.2**

Build Settings -> Validate Workspace -> YES

Localization

In order to use the localization in **AliveCorKitLite** you must add localization to your project. To utilize one of the existing languages that **AliveCorKitLite** supports, you will need to add that same language to your project.

For example, if you wish to use French, you will need to add French in **Xcode->PROJECT->Localizations**, and make sure that the file **fr.lproj/Localizable.strings** exists in your project directory.

Currently **AliveCorKitLite** supports the following languages:

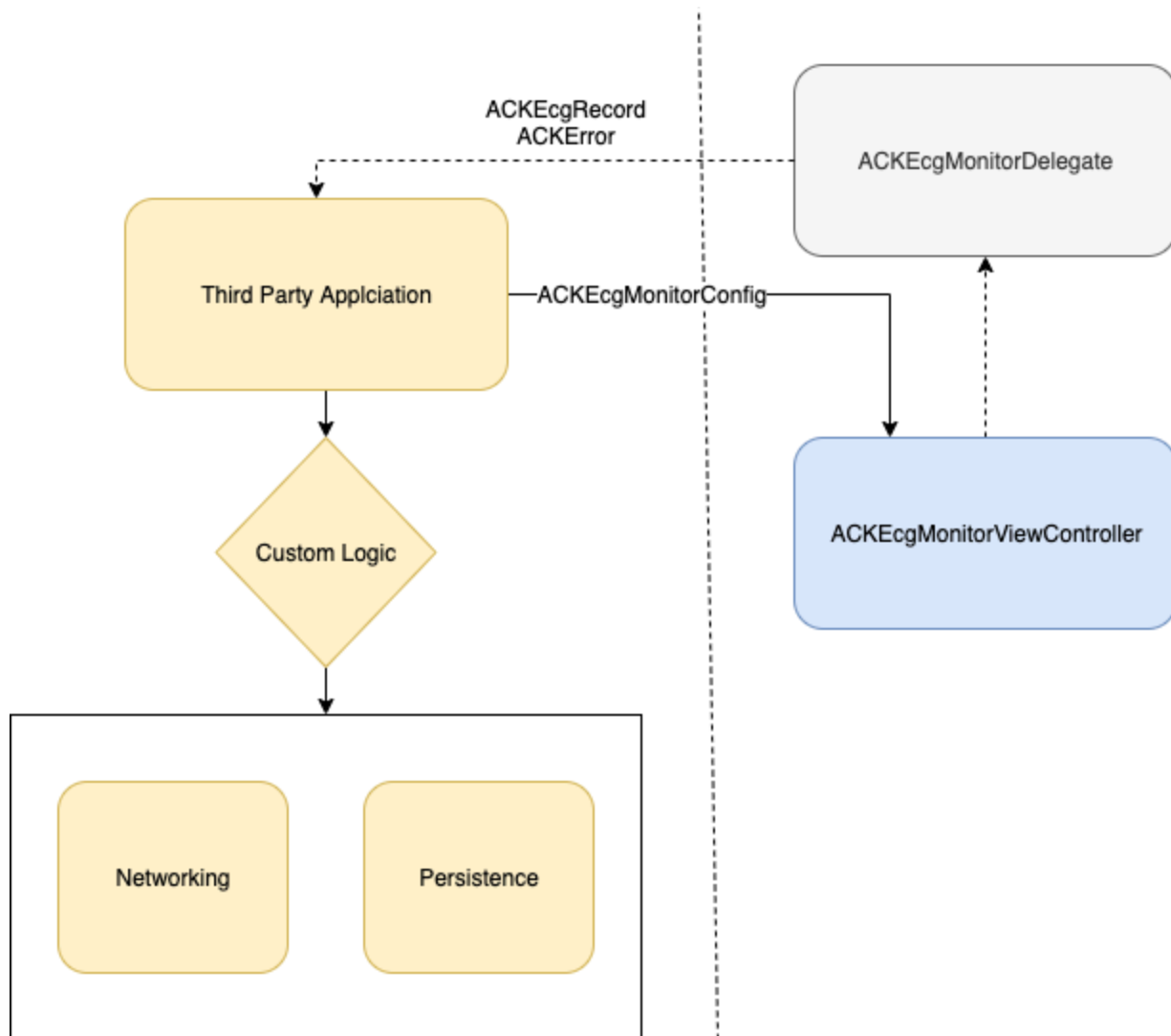
Arabic
Bokmål
Bulgarian
Chinese [Han (Simplified)]
Chinese [Han (Traditional)]
Croatian
Czech
Danish
Dutch
English (United Kingdom)
English (United States)
Finnish
French
German
Greek
Hebrew
Hungarian (Hungary)
Italian
Japanese
Korean
Lithuanian
Malay (Singapore)
Polish
Portuguese (Brazil)

Portuguese (Portugal)
Romanian
Russian
Serbian
Slovak
Spanish
Spanish (United States)
Swedish
Tamil (Singapore)
Thai
Vietnamese
Welsh

Translations are hosted in AliveCorKitAssets, make sure you include it properly as described in the section above.

AliveCorKitLite

AliveCorKitLite provides a set of classes for the ECG recording and AI evaluation using one of the AliveCor devices. With the user's permission, apps communicate with the AliveCorKitLite to process recorded data.



Initializing AliveCorKitLite

ACKManager is an entry point for the SDK usage. The consumer must request a jwt token and to use it for the SDK authorization. Check:

NetworkManager.m

```
+ (void)authSDKWithCompletionHandler:(void (^)(NSError *error, ACKConfiguration *config))completionHandler
```

All interactions with the library will fail if the authorization won't be successful.

Request SDK authorization token

Specify: application's bundleId, partnerId, userId in the request.

```
let configuration = URLSessionConfiguration.default
let session = URLSession(configuration: configuration)
let parameters: [String: Any] = [
    "partnerId": "MDNEjo1UtaMNDrLOKEWc9y7nh3tfyso",
    "bundleId": "com.alivecor.aliveecg.clinical.daily",
    "userId": "bbjzevkwi6wnwf75oz67i0jwv"]
let jsonData = try JSONSerialization.data(withJSONObject: parameters,
options: [])

var request = URLRequest(url: url)
request.httpMethod = "POST"
request.setValue("application/json", forHTTPHeaderField: "Content-Type")
request.httpBody = jsonData

let authStr = "xz32ghmw:pwdf3j5k"
if let authData = authStr.data(using: .utf8) {
    let authValue = "Basic \(authData.base64EncodedString())"
    request.setValue(authValue, forHTTPHeaderField: "Authorization")
}

let dataTask = session.dataTask(with: request) { data, response, error in }
dataTask.resume()
```

Authorize SDK

Use the jwt token from the previous step to authorize SDK usage.

```
if let responseObject = try? JSONSerialization.jsonObject(with: data,
options: []) as? [String: Any],
let jwt = responseObject["jwt"] as? String {
    ACKManager.initWithApiKey(jwt, isDebugMode: true) {}
}
```

Start Recording

```
var error: ACKError?
let config = ACKEcgRecordingConfig(deviceType: device, leadsConfig:
leadsConfig, filterType: filterType, maxDuration: maxDuration,
algorithmPackage: "", error: &error)

let monitorViewController = ACKEcgMonitorViewController(config: config!,
delegate: self)
present(monitorViewController, animated: true)
```

Note: token has expiration time, make sure that app is using app to date token, by checking configuration status. You can do it by observing changes of the statusListener. Check ACKManger for more details.

Handling ECG Recording

After ECG recording is completed, the SDK calls **ecgMonitorViewController:didCompleteRecording:** of the **ACKEcgMonitorDelegate** with **ACKEcgRecord** which holds information about ECG recording. Uploading/Saving ECG records may be implemented in this method. To show ECG results preview you may use either **ACKEcgPreviewViewController** or custom Results screen.

```
func ecgMonitorViewController(_ viewController:ACKEcgMonitorViewController,
didCompleteRecording record: ACKEcgRecord?) {
    guard let record = record else {
        return
    }
    guard let vc = ACKEcgPreviewViewController(record: record) else {
        return
    }
    vc.delegate = self
    // Present ACKEcgPreviewViewController if needed
}
```

Display Legal Information Screen of AliveCorKitLite framework

ACKAboutViewController contains legal, license, and library information and that can be presented through:

```
let vc = ACKAboutViewController()  
self.present(vc, animated: true)
```

Customize Language in AliveCorKitLite framework

We can set the language in EKG recording screen different from the device language.

```
ACKManager.sharedInstance().defaultLanguageType = .chineseTraditional
```

Also, we can set the language of PDF to a language that is different from the EKG recording screen.

E.g.

```
let config = ACKPDFConfig(ecg: record)  
// Setting PDF language to a different language from the system or the  
// recording language  
config.displaySettings?.languageType = DebugUtils.pdfLanguageType  
  
let controller = ACKPDFInteractionController(config: config)  
controller.delegate = self  
controller.presentPdfPreview(for: config, on: pdfOwnerViewController!,  
    supportsLandscape: false, showEncryptionPrompt: false, encryptionRequired:  
    false)
```

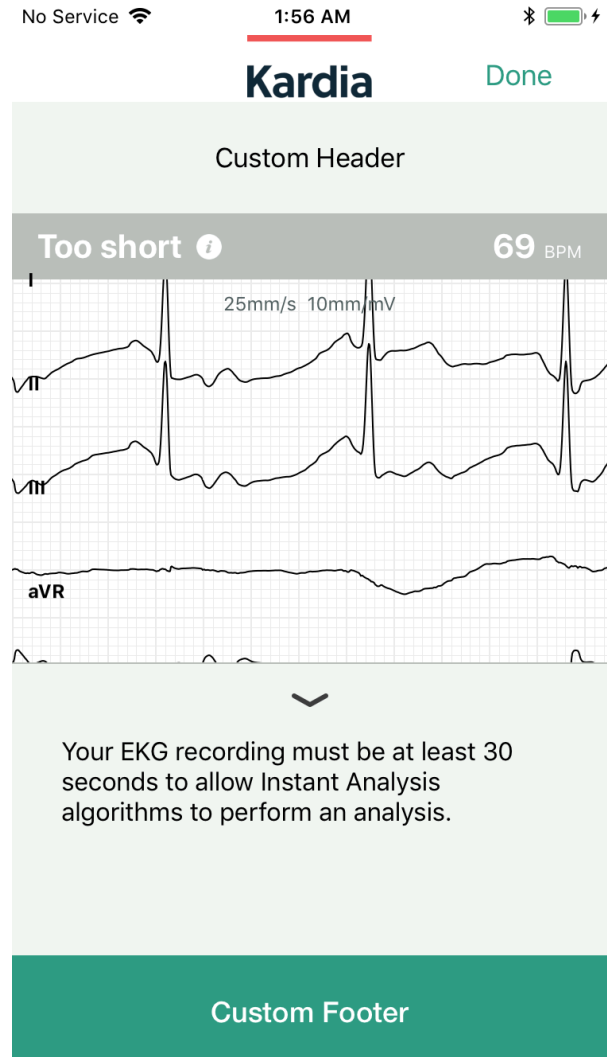
Note:

If we set the defaultLanguageType in ACKManager, it'll apply the language to all strings returning from the framework including the PDF.

Customize Preview Screen

Header & Footer

Use **ACKEcgPreviewViewController** and adopt **ACKEcgPreviewDelegate**, make sure that header/footer view defines height constraint either in xib or programmatically.



Navigation Bar

Use the **ACKEcgPreviewDelegate** methods to add a custom right or left button to the Navigation Bar. These methods supersede the [showDoneButtonInEcgPreviewViewController:](#) method, which corresponds to the the right button, and the [showCancelButtonInEcgPreviewViewController:](#) which corresponds to the the left button. So if

you provide either of the following delegate methods, your custom buttons will be used, even if the [show{Done/Cancel}ButtonInEcgPreviewViewController](#) is defined.

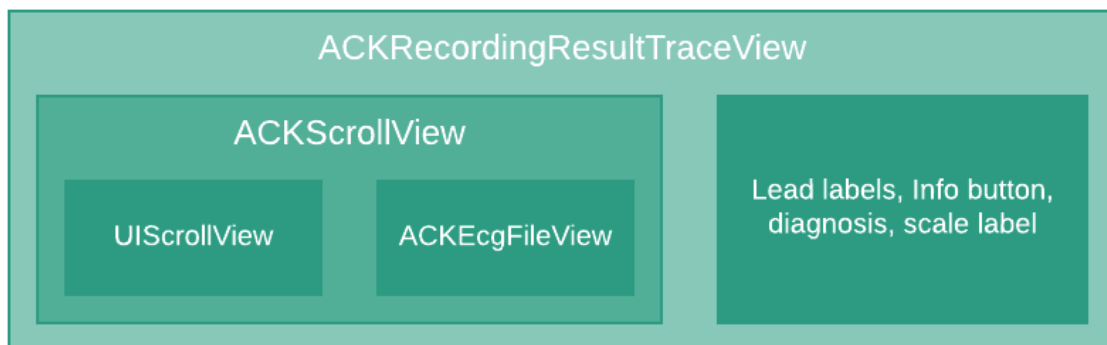
Custom Preview Screen

In order to create a custom view with ECG results, you may either to parse data from `ACKEcgRecord` and to define absolutely custom approach for displaying information or you may use one of the View Classes provided by the SDK: **`ACKEcgFileView`**, **`ACKScrollView`**, **`ACKRecordingResultTraceView`**.

`ACKEcgFileView` - base class for rendering ECG records, other classes are using this class to render ECG.

`ACKScrollView` - renders ECG on `UIScrollView` in order to provide a scrolling functionality.

`ACKRecordingResultTraceView` - adds Leads labels, info button, diagnosis information. This is exactly the view we are using to show ECG on **`ACKEcgPreviewViewController`**



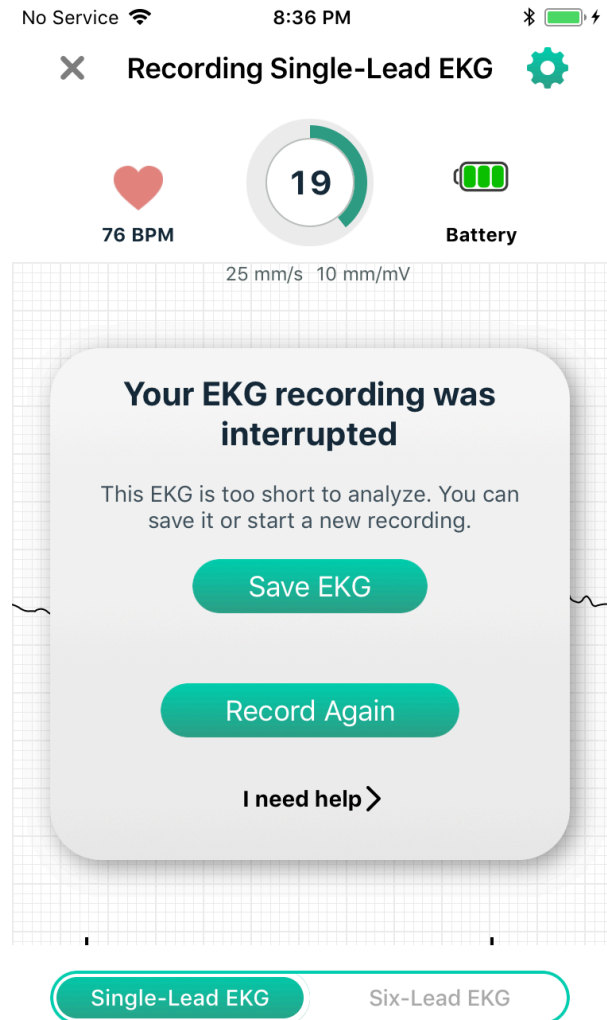
Displaying Custom Records

If you need to use `ACKPreviewViewController` or one of the view classes provided by SDK, you'll need to create a mapper from Json/Custom class to `ACKEcgRecord`. Files associated with record are kept in [ecgFilesDirectory](#) from [ACKManager](#) Paths that are associated with original

and enhanced ecg records are different only by [uuid](#). If you use one of SDKs classes for viewing ECG, you need to ensure that files exist at those paths otherwise the information will not be rendered.

Customizing HUD links

In order to customize web links in the app, developers need to create an instance of the [ACKWebLinks](#) class and to update the [ACKManager](#) with the instance of this class.



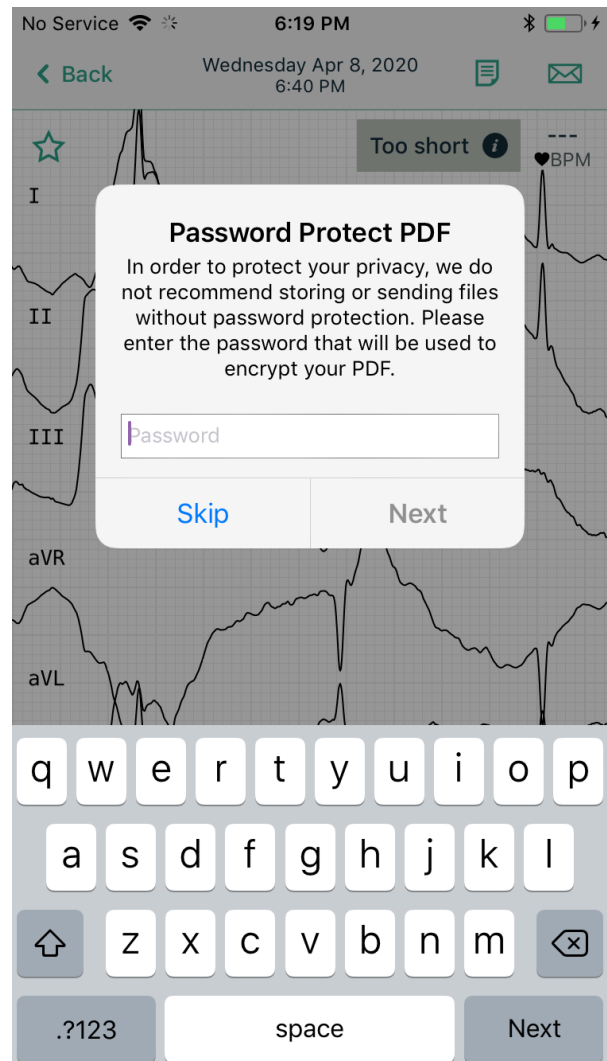
```
let links = ACKWebLinks()
links.tooShort = "https://www.alivecor.com"
ACKManager.sharedInstance().links = links
```

PDF

Built-in PDF generation flow

The SDK provides built-in PDF generation by using `ACKEcgInteractionController`. `ACKEcgInteractionController` generates a PDF report for the predefined `ACKECGRecord` and shows it to the user in `ACKPDFPreviewController`.

The process of the PDF generation is managed by `ACKPDFInteractionControllerDelegate`



PDF generation

The SDK provides functionality to generate PDF reports from predefined `ACKEcgRecord`.

```
let config = ACKPDFConfig(ecg: record)
let pdfPath = ACKPDFReport.pdfPath(for: config)
```

PDF Preview

The SDK provides functionality to show the PDF preview.

```
let config = ACKPDFConfig(ecg: record)
let meta = ACKPDFMetadata()
meta.notes = "Your note are here"
meta.tags = "additional tag are here!!!"
config.metadata = meta
let displaySettings = config.displaySettings
displaySettings?.languageType = DebugUtils.pdfLanguageType
guard let pdfPath = ACKPDFReport.pdfPath(for: config),
    let uuid = record.uuid else {
    return nil
}
let previewController =
    ACKPDPPreviewController.viewController(withPdfFilePath: pdfPath,
        navigationBarTitle: uuid, supportsLandscape: true)
present(previewController, animated: true)
```

Classes

ACKManager

This class is an entry point for the SDK usage both for AliveCorKitLite and AliveCorKit. The consumer must initialize this class during app startup with the partner's API key. Partner's API key is used to validate the authenticity of the partner. All interactions with the library will fail if this class will not be initialized.

Properties

```
@property (nonatomic, readonly) NSString *version;
```

Semantic version of the SDK (e.g., "1.2.3").

```
@property (nonatomic, nullable) ACKLanguageType defaultLanguageType;
```

optional default language for SDK UI/content.

```
@property (nonatomic, readonly) NSString *buildNumber;
```

Internal build number of the SDK.

```
@property (nonatomic, readonly) NSString *apiKey;
```

The API key currently used to authorize SDK access.

```
@property (nonatomic, readonly) NSString *appName;
```

Host application name.

```
@property (nonatomic, readonly) NSString *ecgFilesDirectory;
```

Absolute path to the directory where recorded ECG files are stored.

```
@property (nonatomic, readonly) ACKConfiguration *configuration;
```

Provisioned configuration and consumer-specific restrictions.

```
@property (nonatomic) ACKWebLinks *links;
```

Web links used by HUD screens. Assign to customize.

```
@property (nonatomic, readonly) BOOL isDebugMode;
```

Indicates whether the SDK was initialized in debug (staging) mode.

```
@property (nonatomic, readonly) NSArray<ACKDeviceType> *supportedDevices;
```

Device types supported per current configuration.

Methods

```
+ (instancetype)initWithApiKey:(NSString *)apiKey  
    isDebugMode:(BOOL)isDebugMode  
    statusListener:(nullable ACKStatusListener)statusListener;
```

Initializes the library with the provided API key and listens for changes in the SDK's provisioning state. If the API key is valid and SDK usage is authorized, a configuration object will be returned to the listener; otherwise, an error object will be emitted. When `isDebugMode = true`, the app connects to the staging environment. If you plan to perform UI updates within the callback, ensure that they are executed on the main thread.

apiKey	Partner API key.
isDebugMode	YES to use the staging environment.
statusListener	Completion block invoked once with either a config on success or an error on failure. May be called on a background queue.
return	The shared instance.

```
+ (instancetype)initWithApiKey:(NSString *)apiKey  
    isDebugMode:(BOOL)isDebugMode  
    appName:(NSString *)appName  
    statusListener:(nullable ACKStatusListener)statusListener;
```

Initializes the library with the provided API key and listens for changes in the SDK's provisioning state. If the API key is valid and SDK usage is authorized, a configuration object will be returned to the listener; otherwise, an error object will be emitted. When `isDebugMode = true`, the app connects to the staging environment. If you plan to perform UI updates within the callback, ensure that they are executed on the main thread.

apiKey	Partner API key.
isDebugMode	YES to use the staging environment.
appName	Host application name.
statusListener	Completion block invoked once with either a config on success or an

	error on failure. May be called on a background queue.
return	The shared instance.

- (void)updateApiKey:(NSString *)apiKey;

Update the API key (e.g., when a token is refreshed).

apiKey	New partner API key.
--------	----------------------

- (void)forgetDevice;

Forget the currently paired device and clear persisted pairing data.

- (NSString *)algorithmPackageForCurrentKAI;

Returns the identifier for the currently bundled Kardia AI algorithm package.

- (BOOL)isKAIV2enabled;

Indicates whether Kardia AI v2 is enabled in the current configuration.

- (void)checkStatusWithListener:(nullable ACKStatusListener)statusListener;

Connects to the server and authorizes SDK usage. This method can be used to check the authorization state asynchronously. If the SDK cannot be provisioned, the client app will receive an error object; otherwise, it will receive a configuration object containing the list of allowed devices and the authorization expiration date. If you plan to perform UI updates within the callback, ensure that they are executed on the main thread.

statusListener	Completion block invoked with either a config on success or an error on failure. May be called on a background queue.
----------------	---

ACKKardiaAI

Provides ECG evaluation functionality using Kardia AI.

This class offers APIs for analyzing ECG recordings using Kardia AI. Input samples can be provided either as double-precision millivolt values or as 16-bit signed integers. The data is copied into internal storage during evaluation.

Methods

+ (instancetype)initWithError:(NSError * _Nullable *)error;	
Creates a new instance of Kardia AI evaluator.	
error	Optional pointer to an NSError object that will contain initialization error information if the creation fails.
return	A new ACKKardiaAI instance, or nil if initialization fails.

- (ACEcgEvaluation *)evaluateMVSamples:(const double *)samplesMV length:(NSUInteger)length sampleRate:(ACKSampleRate)sampleRate frequency:(ACKMainsFrequency)frequency algorithmPackage:(NSString *)algorithmPackage;	
Evaluates an ECG signal using an array of double-precision millivolt samples.	
samplesMV	Pointer to an array of ECG samples in millivolts. The data will be copied into internal storage.
length	Number of samples in the array.
sampleRate	Sampling rate of the ECG signal.
frequency	Power line frequency at which the ECG was recorded. One of {ACKMainsFilter50Hz, ACKMainsFilter60Hz}.
algorithmPackage	Specify "kaiv2" to enable Kardia AI v2 if supported. If v2 is unavailable, the SDK automatically falls back to AI v1.
return	An `ACEcgEvaluation` object containing the analysis results

- (ACEcgEvaluation *)evaluateATCSamples:(const short *)samplesATC length:(NSUInteger)length	
--	--

<code>sampleRate:(ACKSampleRate)sampleRate frequency:(ACKMainsFrequency)frequency algorithmPackage:(NSString *)algorithmPackage;</code>	
Evaluates an ECG signal using an array of 16-bit signed integer samples	
samplesATC	Pointer to an array of 16-bit signed ECG samples. The data will be copied into internal storage.
length	Number of samples in the array.
sampleRate	Sampling rate of the ECG signal.
frequency	Power line frequency at which the ECG was recorded. One of {ACKMainsFilter50Hz, ACKMainsFilter60Hz}.
algorithmPackage	Specify "kaiv2" to enable Kardia AI v2 if supported. If v2 is unavailable, the SDK automatically falls back to AI v1.
return	An `ACECGEvaluation` object containing the analysis results

ACECGRecordingConfig

Configuration that defines how the ECG recording flow looks and behaves.

Instances of this object encapsulate the input parameters that drive the ECG monitor's view logic and computed properties (e.g., duration limits, filter, display options).

Unspecified parameters use sensible defaults noted below.

None

```

ACKError *error;
ACECGRecordingConfig *config = [[ACECGRecordingConfig alloc]
initWithDeviceType:device leadsConfig:leadsConfig filterType:filterType
maxDuration:maxDuration algorithmPackage:@" " error:&error];

```

Properties

```
@property (nonatomic, readonly) ACKDeviceType deviceType;
```

Device type used to record ECG.

`@property (nonatomic, readonly) ACKLeadsConfig leadsConfig;`

Recording lead configuration applied to the device.

`@property (nonatomic, readonly) ACKFilterType filterType;`

Filter setting for real-time ECG preview.

`@property (nonatomic, readonly) NSInteger maxDuration;`

Maximum recording duration (seconds) the monitor should allow.

`@property (nonatomic, readonly) NSInteger minDuration;`

Minimum recording duration (seconds).

If the recorded ECG is shorter than this value, the monitor resets its state and the user must record again. The default is 10 seconds.

`@property (nonatomic, readonly) NSInteger frequency;`

Power-line (mains) frequency used for filtering.

One of {ACKMainsFilter50Hz, ACKMainsFilter60Hz}. Stored as an integer; see ACKMainsFrequency for symbolic values.

`@property (nonatomic, readonly) NSInteger sampleRate;`

Sampling rate of the ECG signal (Hz).

Stored as an integer; see ACKSampleRate for common values. The default is 300 Hz.

`@property (nonatomic) BOOL hideTrace;`

Whether to hide the ECG trace during recording.

This flag is ignored for ACKDeviceTypeTriangle.

`@property (nonatomic, copy) NSString *deviceDisplayName;`

Optional override for how the device name is presented to the user.

`@property (nonatomic) BOOL shouldCaptureAudio;`

Whether the application should enable recording of audio notes.

`@property (nonatomic) NSURL *instructionsURL;`

URL of an .mp4 instructions video to present to the user.

`@property (nonatomic) NSString *instructionsMessage;`

Localized instructions message to accompany the recording flow.

```
@property (nonatomic, readonly) NSString *algorithmPackage;
```

Algorithm package used for ECG evaluation.

Specify "kaiv2" to request Kardia AI v2 when eligible; falls back to AI v1 if v2 is not available for the SDK.

```
@property (nonatomic, readonly) BOOL displayBpmDuringRecording;
```

Whether BPM should be displayed during recording.

Methods

```
- (nullable instancetype)initWithDeviceType:(ACKDeviceType)deviceType
                                leadsConfig:(ACKLeadsConfig)leadsConfig
                                filterType:(ACKFilterType)filterType
                                maxDuration:(NSInteger)maxDuration
                                frequency:(NSInteger)frequency
                                sampleRate:(NSInteger)sampleRate
                                minDuration:(NSInteger)minDuration
                                algorithmPackage:(NSString *)algorithmPackage
                                displayBpmDuringRecording:(BOOL)displayBpmDuringRecording
                                error:(NSError *_Nullable)
*_Nullable)error;
```

Create a configuration for the ECG monitor.

deviceType	Hardware device used for recording.
leadsConfig	Recording lead configuration.
filterType	Real-time preview filter.
maxDuration	Maximum duration (seconds). Default if not specified: 30.
frequency	Mains frequency; one of {ACKMainsFilter50Hz, ACKMainsFilter60Hz}. Default if not specified: ACKMainsFrequency60Hz.
sampleRate	Sampling rate (Hz). Default if not specified: 300.
minDuration	Minimum duration (seconds). Default: 10. If the ECG is shorter than this value, the monitor resets.
algorithmPackage	"kaiv2" to request Kardia AI v2 when eligible; falls back to AI v1

	otherwise.
displayBpmDuringRecording	Whether BPM is shown on the recording screen.
error	Populated if initialization fails (e.g., invalid options or API key).
return	A configured instance, or nil on failure.

```

- (nullable instancetype)initWithDeviceType:(ACKDeviceType)deviceType
    leadsConfig:(ACKLeadsConfig)leadsConfig
    filterType:(ACKFilterType)filterType
    maxDuration:(NSInteger)maxDuration
    frequency:(NSInteger)frequency
    sampleRate:(NSInteger)sampleRate
    minDuration:(NSInteger)minDuration
    algorithmPackage:(NSString *)algorithmPackage
    error:(NSError *_Nullable)
    error:(NSError *_Nullable)error;

```

Create a configuration for the ECG monitor.

deviceType	Hardware device used for recording.
leadsConfig	Recording lead configuration.
filterType	Real-time preview filter.
maxDuration	Maximum duration (seconds). Default if not specified: 30.
frequency	Mains frequency; one of {ACKMainsFilter50Hz, ACKMainsFilter60Hz}. Default if not specified: ACKMainsFrequency60Hz.
sampleRate	Sampling rate (Hz). Default if not specified: 300.
minDuration	Minimum duration (seconds). Default: 10. If the ECG is shorter than this value, the monitor resets.
algorithmPackage	"kaiv2" to request Kardia AI v2 when eligible; falls back to AI v1 otherwise.
error	Populated if initialization fails (e.g., invalid options or API key).
return	A configured instance, or nil on failure.

```
- (nullable instancetype)initWithDeviceType:(ACKDeviceType)deviceType
    leadsConfig:(ACKLeadsConfig)leadsConfig
    filterType:(ACKFilterType)filterType
    maxDuration:(NSInteger)maxDuration
    frequency:(NSInteger)frequency
    algorithmPackage:(NSString *)algorithmPackage
    error:(NSError *_Nullable)
    *_Nullable)error;
```

Create a configuration for the ECG monitor.

deviceType	Hardware device used for recording.
leadsConfig	Recording lead configuration.
filterType	Real-time preview filter.
maxDuration	Maximum duration (seconds). Default if not specified: 30.
frequency	Mains frequency; one of {ACKMainsFilter50Hz, ACKMainsFilter60Hz}. Default if not specified: ACKMainsFrequency60Hz.
algorithmPackage	"kaiv2" to request Kardia AI v2 when eligible; falls back to AI v1 otherwise.
error	Populated if initialization fails (e.g., invalid options or API key).
return	A configured instance, or nil on failure.

```
- (nullable instancetype)initWithDeviceType:(ACKDeviceType)deviceType
    leadsConfig:(ACKLeadsConfig)leadsConfig
    filterType:(ACKFilterType)filterType
    maxDuration:(NSInteger)maxDuration
    algorithmPackage:(NSString *)algorithmPackage
    error:(NSError *_Nullable)
    *_Nullable)error;
```

Create a configuration for the ECG monitor.

deviceType	Hardware device used for recording.
leadsConfig	Recording lead configuration.

filterType	Real-time preview filter.
maxDuration	Maximum duration (seconds). Default if not specified: 30.
algorithmPackage	"kaiv2" to request Kardia AI v2 when eligible; falls back to AI v1 otherwise.
error	Populated if initialization fails (e.g., invalid options or API key).
return	A configured instance, or nil on failure.

```
- (nullable instancetype)initWithDeviceType:(ACKDeviceType)deviceType
                                leadsConfig:(ACKLeadsConfig)leadsConfig
                                filterType:(ACKFilterType)filterType
                                algorithmPackage:(NSString *)algorithmPackage
                                error:(NSError
*_Nullable*_Nullable)error;
```

Create a configuration for the ECG monitor.

deviceType	Hardware device used for recording.
leadsConfig	Recording lead configuration.
filterType	Real-time preview filter.
algorithmPackage	"kaiv2" to request Kardia AI v2 when eligible; falls back to AI v1 otherwise.
error	Populated if initialization fails (e.g., invalid options or API key).
return	A configured instance, or nil on failure.

```
- (nullable instancetype)initWithDeviceType:(ACKDeviceType)deviceType
                                algorithmPackage:(NSString *)algorithmPackage
                                error:(NSError *_Nullable
*_Nullable)error;
```

Create a configuration for the ECG monitor.

deviceType	Hardware device used for recording.
------------	-------------------------------------

algorithmPackage	"kaiv2" to request Kardia AI v2 when eligible; falls back to AI v1 otherwise.
error	Populated if initialization fails (e.g., invalid options or API key).
return	A configured instance, or nil on failure.

ACKConfiguration

Represents SDK configuration and partner-specific restrictions.

`ACKConfiguration` contains metadata received from the AliveCor provisioning service, such as authorization expiration, supported device entitlements, and Kardia AI version eligibility. This object helps determine which SDK features are active for a specific partner and when configuration refreshes are required.

Properties

`var expirationDate: Date?`

The date when the SDK authorization expires.

`var features: [FeatureModel]?`

The set of feature entitlements returned by the provisioning service.

`var supportedDevices: [ACKDeviceType]`

The list of devices supported for the current partner configuration.

`var isStale: Bool`

Indicates whether the configuration should be refreshed.
Returns `true` if the phone-home interval has elapsed, or `false` otherwise.

`var algVersionType: ACKAlgorithmVersion`

The enumerated type of the current algorithm package.
Returns `.KAIV2` if `algVersion` equals `"kaiv2"` (case-insensitive), otherwise `.KAIV1`.

`var hasExpired: Bool`

Indicates whether the configuration has expired.

Returns `true` if the current authorization is no longer valid, or `false` if it is still active or authorization checks are disabled.

ACKDevice

Represents metadata about the physical device used to record an ECG.

This class encapsulates identifying and diagnostic information for AliveCor ECG devices, such as hardware/firmware revisions, serial numbers, and battery level. Availability of certain fields depends on the device type.

Properties

```
@property (nonatomic, readonly) ACKDeviceType deviceType;
```

The hardware type used to record the ECG.

```
@property (nonatomic, copy, readonly, nullable) NSString *hardwareRevision;
```

Hardware revision identifier.

Available only for `ACKDeviceTypeTriangle` and `ACKDeviceTypeCard`.

```
@property (nonatomic, copy, readonly, nullable) NSString *firmwareRevision;
```

Firmware revision identifier.

Available only for `ACKDeviceTypeTriangle` and `ACKDeviceTypeCard`.

```
@property (nonatomic, copy, readonly, nullable) NSString *serialNumber;
```

Device serial number.

Available only for `ACKDeviceTypeTriangle` and `ACKDeviceTypeCard`.

```
@property (nonatomic, copy, readonly, nullable) NSString *bluetoothId;
```

Bluetooth identifier (if applicable).

Available only for Bluetooth-capable devices.

```
@property (nonatomic, copy, readonly, nullable) NSString *deviceName;
```

Human-readable device name.

```
@property (nonatomic, readonly) CGFloat batteryLevel;
```

Current battery level (0–100).

Available only for `ACKDeviceTypeTriangle` and `ACKDeviceTypeCard`.

```
@property (nonatomic, copy, readonly, nullable) NSString *uuid;
```

Unique device identifier (UUID string).

Available only for `ACKDeviceTypeTriangle` and `ACKDeviceTypeCard`.

Methods

```
- (instancetype)initWithDeviceType:(ACKDeviceType)deviceType;
```

Initializes a new device instance with the specified device type.

```
- (instancetype)initWithDeviceType:(ACKDeviceType)deviceType
    hardwareRevision:(nullable NSString *)hardwareRevision
    firmwareRevision:(nullable NSString *)firmwareRevision
    serialNumber:(nullable NSString *)serialNumber
    batteryLevel:(CGFloat)batteryLevel
    uuid:(nullable NSString *)uuid
    bluetoothId:(nullable NSString *)bluetoothId
    deviceName:(nullable NSString *)deviceName;
```

Initializes a new device instance with the specified hardware metadata.

```
- (BOOL)isTriangleDeviceVariant;
```

Returns `YES` if the receiver represents a BLE variant of a Triangle device.

```
- (BOOL)isBLEDevice;
```

Returns `YES` if the device supports Bluetooth Low Energy (BLE) communication.

```
+ (BOOL)isTriangleDeviceVariantType:(ACKDeviceType)deviceType;
```

Returns `YES` if the receiver represents a BLE variant of a Triangle device.

```
+ (BOOL)isBLEDeviceType:(ACKDeviceType)deviceType;
```

Returns `YES` if the given type supports Bluetooth Low Energy (BLE).

```
+ (NSString *)deviceNameFromType:(ACKDeviceType)type;
```

Returns the default display name for the given device type.

ACKEcgEvaluation

Represents the results of an ECG evaluation performed by Kardia AI.

This class encapsulates diagnostic information derived from an ECG analysis, including algorithm determination, modifiers, calculated metrics such as heart rate, and localized diagnostic descriptions. It also includes metadata such as algorithm version and package identifiers.

Properties

```
@property (nonatomic, copy, readonly) NSString *version;
```

The version of the Kardia AI library used for this evaluation.

```
@property (nonatomic, copy, readonly) NSString *algorithmPackage;
```

The algorithm package used for the evaluation (e.g., `"kaiv2"`).

```
@property (nonatomic, copy, readonly) ACKAlgorithmDetermination determination;
```

The primary determination result produced by Kardia AI.

```
@property (nonatomic, copy, readonly) ACKDeterminationModifier modifier;
```

Additional modifier describing characteristics of the detected rhythm.

```
@property (nonatomic, readonly) CGFloat averageHeartRate;
```

The average heart rate measured during the recording, in beats per minute (bpm). The valid range is 30–220 bpm. If the detected heart rate falls outside these bounds, or the signal quality is too poor to provide a reliable estimate, the returned value is `0`.

```
@property (nonatomic, readonly) NSArray *errors;
```

An array of errors encountered during evaluation, if any.

`@property (nonatomic) BOOL isInverted;`

Indicates whether the ECG signal was inverted.

`@property (nonatomic, copy, nullable) NSArray *averageBeats;`

Average beat data provided by Kardia AI.

`@property (nonatomic, copy, nullable) NSArray *beats;`

Detected individual beats with annotations, as calculated by Kardia AI.

`@property (nonatomic, copy, nullable) NSArray *intervals;`

R–R interval values provided by Kardia AI.

Methods

```
- (instancetype)initWithKaiVersion:(NSString *)version
    algorithmPackage:(NSString *)algorithmPackage
    determination:(NSString *)determination
    modifier:(NSString *)modifier
    averageHeartRate:(CGFloat)averageHeartRate
    isInverted:(BOOL)isInverted
    errors:(NSArray * _Nullable)errors;
```

Initializes an evaluation result with the given Kardia AI data.

version	The Kardia AI version.
algorithmPackage	The algorithm package used for evaluation.
determination	The algorithm’s primary determination result.
modifier	The determination modifier.
averageHeartRate	The average heart rate in bpm
isInverted	Whether the ECG trace is inverted.
errors	Any errors generated during evaluation.
return	A new instance of `ACEKecgEvaluation`.

- (NSString *)determinationLabel;

Returns a short, human-readable label for the current determination.
Example: "Sinus Rhythm", "Unclassified".

- (NSString *)determinationWithModifierLabel;

Returns a short, human-readable label combining the determination and modifier.
Example: "Normal Sinus Rhythm", "Sinus Rhythm with VEBs".

- (NSString *)localizedDeterminationShortTitle;

Returns the localized short title for the diagnosis.
Example: "Normal", "Too Short".

- (NSString *)localizedDeterminationShortTitle:(nullable
ACKLanguageType)type;

Returns the localized short title for the diagnosis in the specified language.

- (NSString *)localizedDescription;

Returns the full localized diagnostic description.
Includes the analysis summary, disclaimer, and any additional information.
Example:
"No rhythm abnormalities detected in your EKG. Kardia cannot detect signs of a heart attack.
If you believe you are having a medical emergency, call emergency services."

- (NSString *)localizedDescription:(nullable ACKLanguageType)type;

Returns the full localized diagnostic description in the specified language.

- (NSString *)localizedDeterminationDescription;

Returns the localized algorithm determination description.
Example: "No rhythm abnormalities detected in your EKG."

- (NSString *)localizedDeterminationDescription:(nullable ACKLanguageType)type;

Returns the localized algorithm determination description in the specified language.

- (NSString *)localizedDisclaimer;

Returns the localized disclaimer text for the analysis.
Example:
"Kardia cannot detect signs of a heart attack. If you believe you are having a medical emergency, call emergency services. DO NOT change your medication without talking to your doctor."

- (NSString *)localizedDisclaimer:(nullable ACKLanguageType)type;

Returns the localized disclaimer text for the specified language.

- (NSString *)localizedAdditionalInformation;

Returns the localized additional information for this analysis result.
Example:
"Atrial fibrillation was not detected and your EKG does not fall under the classifications of Normal, Bradycardia, or Tachycardia. This may be caused by other arrhythmias, unusually fast or slow heart rates, or poor quality recordings."

- (NSString *)localizedAdditionalInformation:(nullable ACKLanguageType)type;

Returns the localized additional information for the specified language.

- (NSString *)localizedDeterminationTitle;

Returns the localized descriptive title of the algorithm result.
Example: "Normal", "Your EKG recording was interrupted".

- (UIColor *)determinationColor;

Returns the color associated with this determination result.

+(ACKAlgorithmDetermination)ecgAlgorithmDeterminationForAfibDetected:(BOOL)afibDetected nsrDetected:(BOOL)nsrDetected noiseDetected:(BOOL)noiseDetected durationMilliseconds:(NSInteger)durationMilliseconds;

Maps a set of legacy boolean flags to a modern algorithm determination value.

afibDetected	The legacy AFib detection flag.
nsrDetected	The legacy Normal Sinus Rhythm detection flag.
noiseDetected	The legacy noise detection flag.
durationMilliseconds	The duration of the ECG recording, in milliseconds.
return	The corresponding `ACKAlgorithmDetermination` value.

ACKDeviceType

String-backed enumeration for the device type used for ECG recording.

Possible values	
ACKDeviceTypeUnknown	The unknown device.
ACKDeviceTypeTriangle	The Triangle device (retail name KardiaMobile 6L).
ACKDeviceTypeMobile	The KardiaMobile device.

ACKDeviceTypeBand	The Kardia Band.
ACKDeviceTypeSakuraOne	The SakuraOne.

ACKLeadsConfig

Recording lead configuration applied to the device (either single-lead or six-lead).

Possible values	
ACKLeadsConfigSingle	The single leads mode.
ACKLeadsConfigSix	The six leads mode. Available only in the Triangle device.

ACKFilterType

Filter applied to the ECG recording preview.

Possible values	
ACKFilterTypeEnhanced	The enhanced filter.
ACKFilterTypeOriginal	The ECG recording is baseline corrected and mains-filtered.

ACKAlgorithmDetermination

String-backed enumeration of Kardia AI determination codes.
Represents both the determination and its corresponding status code.

Possible values	
ACKAlgorithmDeterminationNoAnalysis	
ACKAlgorithmDeterminationUnreadable	
ACKAlgorithmDeterminationTooShort	

ACKAlgorithmDeterminationTooLong
ACKAlgorithmDeterminationUnclassified
ACKAlgorithmDeterminationNormal
ACKAlgorithmDeterminationAtrialFibrillation
ACKAlgorithmDeterminationBradycardia
ACKAlgorithmDeterminationTachycardia

ACKEcgRecord

Represents a captured ECG recording and its associated context.

`ACEcgRecord` is produced by `ACEcgMonitorViewController` and serves as the primary data model for SDK UI components. It bundles the raw/enhanced ECG file paths, the recording configuration, device metadata, timing information, and any Kardia AI evaluation results.

Properties

`@property (nonatomic, copy, readonly, nullable) NSString *uuid;`

Unique identifier for the ECG recording.

`@property (nonatomic, copy, readonly, nullable) NSString *originalPath;`

Path to the raw ATC recording file (baseline-corrected and mains-filtered).

`@property (nonatomic, copy, readonly, nullable) NSString *enhancedPath;`

Path to the enhanced-filtered ATC recording file.

`@property (nonatomic, copy, readonly, nullable) NSString *audioNotesPath;`

Path to the audio-notes file (M4A format), if captured.

`@property (nonatomic, copy, readonly, nullable) NSDate *recordedAt;`

Measurement date/time of the recording. Not required to render the ECG trace.

`@property (nonatomic, readonly) NSNumber *timeZoneOffset;`

Time-zone offset for the measurement (relative to UTC).

Not required to render the ECG trace.

```
@property (nonatomic, copy, readonly) NSString *filesDirectory;
```

Directory containing the ECG recording files.

```
@property (nonatomic, copy, readonly, nullable) NSNumber *duration;
```

Recording duration in milliseconds.

```
@property (nonatomic, copy, readonly, nullable) ACKDevice *device;
```

Device used to capture the ECG.

```
@property (nonatomic, copy, readonly) ACKEcgRecordingConfig *config;
```

Configuration applied during the recording.

```
@property (nonatomic, copy, readonly, nullable) ACKEcgEvaluation *evaluation;
```

Kardia AI evaluation results for the recording, if available.

```
@property (nonatomic, copy, nullable) NSDictionary *metadata;
```

Placeholder for partner-specific metadata.

Methods

```
- (instancetype)initWithUUID:(NSString * _Nullable)uuid
    duration:(NSNumber * _Nullable)duration
    config:(ACKEcgRecordingConfig *)config
    device:(ACKDevice * _Nullable)device
    recordedAt:(NSDate * _Nullable)recordedAt
    timeZoneOffset:(NSNumber * _Nullable)timeZoneOffset
    evaluation:(ACKEcgEvaluation * _Nullable)evaluation;
```

Initializes a record with full context and optional evaluation.

uuid	Unique identifier for the ECG recording.
duration	Recording duration in milliseconds.
config	Configuration used to record the ECG.
device	Device used to record the ECG
recordedAt	Measurement date/time

timeZoneOffset	Time-zone offset relative to UTC.
evaluation	Kardia AI evaluation for this recording.
return	A new `ACEcgRecord` instance.

```
- (instancetype)initWithUUID:(NSString * _Nullable)uuid
    duration:(NSNumber * _Nullable)duration
    config:(ACEcgRecordingConfig *)config
    device:(ACKDevice * _Nullable)device
    recordedAt:(NSDate * _Nullable)recordedAt
    timeZoneOffset:(NSNumber * _Nullable)timeZoneOffset;
```

Initializes a record with full context and optional evaluation.

uuid	Unique identifier for the ECG recording.
duration	Recording duration in milliseconds.
config	Configuration used to record the ECG.
device	Device used to record the ECG
recordedAt	Measurement date/time
timeZoneOffset	Time-zone offset relative to UTC.
return	A new `ACEcgRecord` instance.

```
- (instancetype)initWithUUID:(NSString *)uuid
    ecgPath:(NSString *)ecgPath
    deviceType:(ACKDeviceType)deviceType
    filterType:(ACKFilterType)filterType
    leadsConfig:(ACKLeadsConfig)leadsConfig
    evaluation:(ACEcgEvaluation * _Nullable)evaluation
    recordedAt:(NSDate * _Nullable)recordedAt;
```

Creates a lightweight record for displaying ECG content using a single ATC path. Suitable for `ACEcgFileView`, `ACKScrollView`, and `ACKRecordingResultTraceView`.

uuid	Unique identifier for the ECG recording.
------	--

ecgPath	Path to the ECG ATC file.
deviceType	Device type used to record the ECG.
filterType	Filter applied for rendering.
leadsConfig	Leads configuration used for the recording.
evaluation	Optional Kardia AI evaluation.
recordedAt	Optional measurement date/time.
return	A new `ACEcgRecord` instance.

```
- (instancetype)initWithEcgPath:(NSString *)ecgPath
                        deviceType:(ACKDeviceType)deviceType
                        filterType:(ACKFilterType)filterType
                        leadsConfig:(ACKLeadsConfig)leadsConfig
                        evaluation:(ACEcgEvaluation * _Nullable)evaluation;
```

Creates a lightweight record (no UUID parameter) for displaying ECG content.
Suitable for `ACEcgFileView`, `ACKScrollView`, and `ACKRecordingResultTraceView`.

ecgPath	Path to the ECG ATC file.
deviceType	Device type used to record the ECG.
filterType	Filter applied for rendering.
leadsConfig	Leads configuration used for the recording.
evaluation	Optional Kardia AI evaluation.
return	A new `ACEcgRecord` instance.

```
- (instancetype)initWithUUID:(NSString *)uuid
                        ecgPath:(NSString *)ecgPath
    enhancedEcgPath:(NSString *)enhancedPath
                        deviceType:(ACKDeviceType)deviceType
                        filterType:(ACKFilterType)filterType
                        leadsConfig:(ACKLeadsConfig)leadsConfig
                        evaluation:(ACEcgEvaluation * _Nullable)evaluation
```

<pre>duration:(NSNumber *)duration recordedAt:(NSDate * _Nullable)recordedAt;</pre>	
Creates a lightweight record with both raw and enhanced ATC paths.	
uuid	Unique identifier for the ECG recording.
ecgPath	Path to the raw/baseline-corrected ATC file.
enhancedPath	Path to the enhanced-filtered ATC file.
deviceType	Device type used to record the ECG.
filterType	Filter applied for rendering.
leadsConfig	Leads configuration used for the recording.
evaluation	Optional Kardia AI evaluation.
duration	Recording duration in milliseconds.
recordedAt	Optional measurement date/time.
return	A new `ACECGRecord` instance.

ACKErrorCode

The error class. Defines error type that occurred while recording or evaluating ECG.

Error types	
ACKErrorCodeNone	Reserved default state.
ACKErrorCodeInvalidLeadsConfig	The provided leads configuration is invalid.
ACKErrorCodeInvalidRecordingConfig	The provided ECG recording configuration is invalid.
ACKErrorCodeInvalidToken	The authorization token is invalid.
ACKErrorCodeDeviceTypeNotSupported	The specified device type is not supported.
ACKErrorCodeBluetoothOff	Bluetooth is turned off. (Triangle and Kardia Card only.)

ACKErrorCodeBluetoothUnsupported	Bluetooth is not supported on this device. (Triangle and Kardia Card only.)
ACKErrorCodeBluetoothDisconnected	The Bluetooth device disconnected. (Triangle and Kardia Card only.)
ACKErrorCodeBluetoothPairingRequestDenied	The Bluetooth pairing request was denied. (Triangle and Kardia Card only.)
ACKErrorCodeMicPermissionEKG	Microphone permission is required to record an EKG.
ACKErrorCodeTriangleBattery	The Bluetooth device battery level is too low to record. (Triangle and Kardia Card only.)
ACKErrorCodeTriangleConnection	Connection to the Triangle device failed. (Triangle only.)
ACKErrorCodeMainsNoiseEarly	Excessive mains noise and the recording is too short to save.
ACKErrorCodeEvaluationInvalidOutput	The Kardia AI evaluation returned invalid output.
ACKErrorCodeEvaluationTooShort	Recording completed, but duration is too short for Kardia AI evaluation.
ACKErrorCodeEvaluationUnreadable	Recording completed, but Kardia AI could not evaluate the ECG.
ACKErrorCodeElectricallyInterfered	Recording completed, but the ECG is too short or unreadable (electrical interference).
ACKErrorCodeNotEnoughStorage	Not enough disk space to record an ECG.
ACKErrorCodeAudioSession	Audio session error while using KardiaMobile (connection or runtime failure).
ACKErrorCodeAudioInsufficientPriority	Insufficient audio session priority for KardiaMobile.
ACKErrorCodeDeviceRegistration	Device registration failed.
ACKErrorCodeDeviceDisabled	Device is registered, but ECG recordings are not allowed.

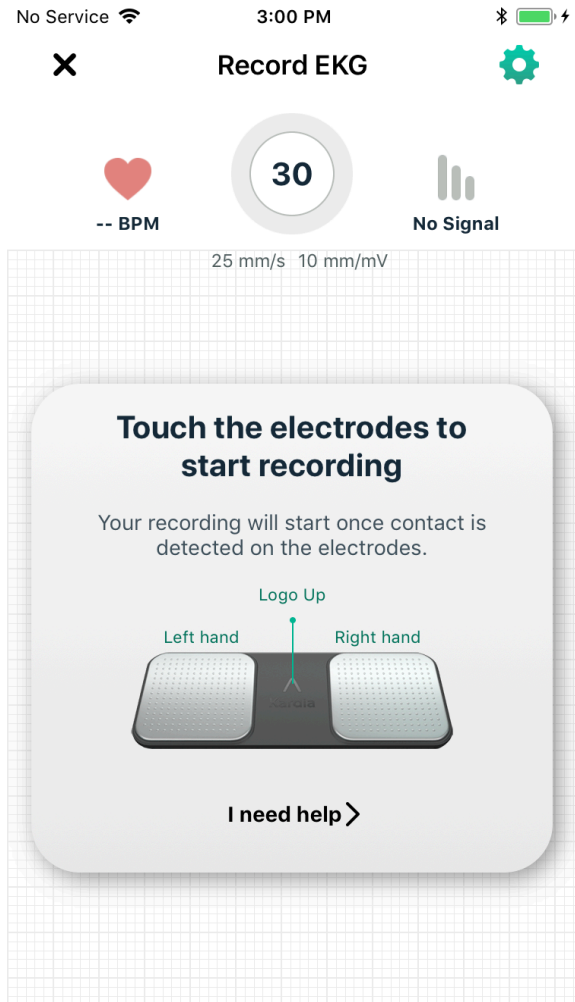
ACKErrorCodeStaleSDKConfiguration	SDK configuration has not been refreshed within the required time window; re-authentication is needed.
ACKErrorCodeStaleDevicePlan	Device plan has not been refreshed within the required time window; re-authentication is needed.
ACKErrorCodeInvalidConfiguration	SDK configuration is invalid or missing required values.
ACKErrorCodeInvalidApiKey	Provided SDK API key is invalid or does not match.
ACKErrorCodeMismatchedUUID	UUIDs of the original and enhanced ATC files do not match.
ACKErrorCodeAtcNotFound	ATC file could not be found.
ACKErrorCodeDeviceNameMismatch	Saved/passed device name does not match the detected device name.

ACKEcgMonitorViewController

A view controller for the ECG recording. Internally manages all steps of the communication with AliveCor devices and AI evaluation. Provides predefined UI and UX for recording ECG.

Responsibilities:

- Pairing with devices.
- Bluetooth communication cycle
- Audio communication cycle
- Kardia AI evaluations
- Error handling.



Properties

```
@property (nonatomic, weak, nullable) id<ACEcgMonitorDelegate> delegate;
```

Delegate that receives monitor events.

Methods

```
- (nullable instancetype)initWithConfig:(ACEcgRecordingConfig *)config  
    delegate:(id<ACEcgMonitorDelegate>)delegate;
```

Creates a monitor with the specified configuration and delegate.

config	Configuration to apply during recording
delegate	Delegate that receives monitor events.
return	A new instance, or nil if initialization fails.

```
- (nullableinstancetype)initWithConfig:(ACKEcgRecordingConfig *)config
    delegate:(id<ACKEcgMonitorDelegate>)delegate
    error:(NSError * _Nullable * _Nullable)error;
```

Creates a monitor with the specified configuration and delegate.

config	Configuration to apply during recording.
delegate	Delegate that receives monitor events.
error	Populated if initialization fails.
return	A new instance, or nil on failure.

```
- (void)stopMonitor;
```

Stops the monitor and releases underlying resources.

Invoke this when providing a custom back/close action instead of dismissing the view controller directly.

ACKEcgMonitorDelegate

Delegate methods for tracking ECG recording progress, handling UI actions, and receiving completion and error callbacks from `ACKEcgMonitorViewController`.

Methods

```
- (void)ecgMonitorViewController:(ACKEcgMonitorViewController *
    _Nonnull)viewController
```

<code>didConnectWithDevice:(ACKDevice * _Nonnull)device continueWithRecording:(void (^ _Nonnull)(BOOL))continueHandler;</code>	
Notifies the delegate that a device is connected and is ready to record.	
viewController	The ECG monitoring view controller.
device	The connected device that will be used for recording.
continueHandler	Call with YES to proceed with recording, or NO to stop the monitor. If NO, it is the caller's responsibility to dismiss the monitor view controller.

<code>- (void)ecgMonitorViewController:(ACEcgMonitorViewController * _Nonnull)viewController didCancelWithError:(nullable NSError *)error;</code>	
Notifies the delegate that the recording was canceled.	
viewController	The ECG monitoring view controller.
error	An error describing why cancellation occurred, or nil if none.

<code>- (void)ecgMonitorViewController:(ACEcgMonitorViewController * _Nonnull)viewController didPressSettingWithConfig:(nullable ACEcgRecordingConfig *)config;</code>	
Notifies the delegate that the Settings button was pressed.	
viewController	The ECG monitoring view controller.
config	The current ECG monitor configuration.

<code>- (void)ecgMonitorViewController:(ACEcgMonitorViewController * _Nonnull)viewController didCompleteRecording:(nullable ACEcgRecord *)record;</code>	
Notifies the delegate that the recording completed successfully.	
viewController	The ECG monitoring view controller.

record	The resulting ECG record.
--------	---------------------------

```
- (void)ecgMonitorViewController:(ACEcgMonitorViewController *
_Nonnull)viewController
    didEncounterError:(nullable NSError *)error;
```

Reports an error encountered during recording.

viewController	The ECG monitoring view controller.
error	The recording error.

```
-
(BOOL)showCancelButtonInEcgMonitorViewController:(ACEcgMonitorViewControll
er * _Nonnull)viewController;
```

Asks the delegate whether the Cancel button should be displayed.

viewController	The ECG monitoring view controller.
return	YES to show the Cancel button; NO otherwise.

```
-
(BOOL)showSettingsButtonInEcgMonitorViewController:(ACEcgMonitorViewContro
ller * _Nonnull)viewController;
```

Asks the delegate whether the Settings button should be displayed.

viewController	The ECG monitoring view controller.
return	YES to show the Settings button; NO otherwise.

```
- (UIBarButtonItem *
_Nullable)rightItemViewForEcgMonitorViewController:(ACEcgMonitorViewContro
ller * _Nonnull)viewController;
```

Requests a custom right bar button item for the navigation bar.

If implemented, this supersedes `showSettingsButtonInEcgMonitorViewController:`.

viewController	The ECG monitoring view controller.
return	The custom right bar button item, or nil to use the default behavior.

```
- (UIBarButtonItem *  
_Nullable)leftItemViewForEcgMonitorViewController:(ACEcgMonitorViewControl  
ler * _Nonnull)viewController;
```

Requests a custom left bar button item for the navigation bar.
If implemented, this supersedes `showCancelButtonInEcgMonitorViewController:`.

viewController	The ECG monitoring view controller.
return	The custom left bar button item, or nil to use the default behavior.

```
- (NSString *  
_Nullable)confirmRecordingMessageForEcgMonitorViewController:(ACEcgMonitor  
ViewController * _Nonnull)viewController;
```

Asks the delegate for additional text to display in the “confirm recording” dialog.

viewController	The ECG monitoring view controller.
return	A message to display in the confirmation dialog, or nil for none.

```
- (void)ecgMonitorViewController:(ACEcgMonitorViewController *  
_Nonnull)viewController didChangeLeadsConfig:(ACKLeadsConfig)config;
```

Notifies the delegate that the lead configuration changed.

viewController	The ECG monitoring view controller.
config	The new leads configuration.

-

```
(BOOL)ecgMonitorViewControllerShouldEnableLeadModeSwitch:(ACEcgMonitorViewController * _Nonnull)viewController;
```

Asks the delegate whether switching lead mode is allowed.

viewController	The ECG monitoring view controller.
----------------	-------------------------------------

return	YES to enable lead-mode switching; NO otherwise.
--------	--

```
- (void)ecgMonitorViewController:(ACEcgMonitorViewController * _Nonnull)viewController  
    connectedDevice:(ACKDevice * _Nonnull)connectedDevice  
    batteryLevelDetected:(NSInteger)batteryLevel;
```

Notifies the delegate of a detected battery level on the connected device.

viewController	The ECG monitoring view controller.
----------------	-------------------------------------

connectedDevice	The connected BLE device.
-----------------	---------------------------

batteryLevel	The detected battery level.
--------------	-----------------------------

return	statement
--------	-----------

```
- (void)ecgMonitorViewController:(ACEcgMonitorViewController * _Nonnull)viewController  
    didEncounterAudioError:(nullable NSError *)error;
```

Reports an audio-related error encountered during recording.

Called only for ultrasound devices (Kardia Mobile, Omron Complete). If not implemented, the SDK shows an alert and exits the recording flow.

viewController	The ECG monitoring view controller.
----------------	-------------------------------------

error	The audio error.
-------	------------------

```
- (NSArray<ACKDeviceType> *  
 _Nonnull)availableDeviceTypes:(ACEcgMonitorViewController *  
 _Nonnull)viewController;
```

Requests the list of paired device types available to the user.	
viewController	The ECG monitoring view controller.
return	An array of paired device types.

<pre>- (void)didPressAddNewDevice:(ACEcgMonitorViewController * _Nonnull)viewController;</pre>	
Notifies the delegate that the “Add New Device” button was tapped.	
viewController	The ECG monitoring view controller.

<pre>- (void)didChangeToDevice:(ACEcgMonitorViewController * _Nonnull)viewController deviceType:(ACKDeviceType _Nonnull)deviceType;</pre>	
Notifies the delegate that the user selected a different device type.	
viewController	The ECG monitoring view controller.
deviceType	The newly selected device type.

<pre>- (void)didChangeToDevice:(ACEcgMonitorViewController * _Nonnull)viewController deviceType:(ACKDeviceType _Nonnull)deviceType;</pre>	
Notifies the delegate that the user selected a different device type.	
viewController	The ECG monitoring view controller.
deviceType	The newly selected device type.

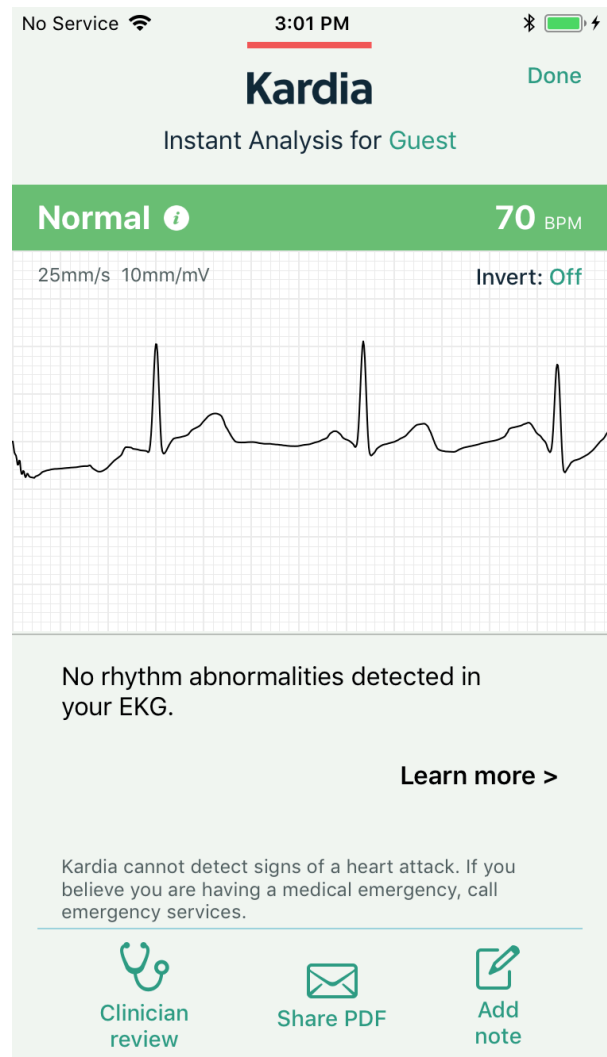
<pre>- (void)ecgMonitorViewController:(ACEcgMonitorViewController * _Nonnull)viewController didReceiveEcgFrame:(struct ECGFrame)ecgFrame;</pre>	
---	--

Provides real-time filtered ECG samples for live preview during recording.	
viewController	The ECG monitoring view controller.
ecgFrame	The most recent samples from each ECG lead.

ACKEcgPreviewViewController

A view controller that displays the results of an ECG recording and its Kardia AI evaluation.

`ACKEcgPreviewViewController` presents the ECG waveform, analysis summary, and related diagnostic results. It supports switching between different filter types for previewing the recorded ECG signal.



Properties

`@property (nonatomic, copy, readonly, nullable) ACEcgRecord *record;`

The ECG record currently displayed in the preview screen.

`@property (nonatomic, weak, nullable) id<ACEcgPreviewDelegate> delegate;`

Use this delegate to respond to user actions (such as tapping Done or Cancel), and to provide custom UI elements (header, footer, or navigation items) for the ECG preview screen.

`@property (nonatomic) ACKFilterType previewFilterType;`

The current preview filter applied to the ECG waveform.

Methods

```
- (nullableinstancetype)initWithRecord:(ACEcgRecord * _Nonnull)record;
```

Initializes a new preview controller with the specified ECG record.

record	The ECG record to be displayed in the preview screen.
--------	---

return	A new instance of `ACEcgPreviewViewController`.
--------	---

ACEcgPreviewDelegate

Defines methods that allow the delegate to manage interactions and customize the appearance of the ECG results screen presented by `ACEcgPreviewViewController`.

Use this protocol to respond to user actions (such as tapping Done or Cancel), and to provide custom UI elements (header, footer, or navigation items) for the ECG preview screen.

Methods

```
- (void)ecgPreviewViewController:(ACEcgPreviewViewController * _Nonnull)viewController  
    didFinishWithRecord:(ACEcgRecord * _Nullable)record;
```

Notifies the delegate that the Done button was tapped.

viewController	The ECG preview view controller.
----------------	----------------------------------

record	The ECG record associated with the completed recording.
--------	---

```
- (void)ecgPreviewViewController:(ACEcgPreviewViewController * _Nonnull)viewController  
    didCancelWithRecord:(ACEcgRecord * _Nullable)record;
```

Notifies the delegate that the Cancel button was tapped.

viewController	The ECG preview view controller.
record	The ECG record associated with the completed recording.

```
-
(BOOL)showDoneButtonInEcgPreviewViewController:(ACEcgPreviewViewController
* _Nonnull)viewController;
```

Asks the delegate whether the Done button should be displayed.

viewController	The ECG preview view controller.
return	YES to display the Done button; NO otherwise.

```
-
(BOOL)showCancelButtonInEcgPreviewViewController:(ACEcgPreviewViewControll
er * _Nonnull)viewController;
```

Asks the delegate whether the Cancel button should be displayed.

viewController	The ECG preview view controller.
return	YES to display the Cancel button; NO otherwise.

```
- (UIView *
_Nullable)footerViewForEcgPreviewViewController:(ACEcgPreviewViewControlle
r * _Nonnull)viewController;
```

Asks the delegate for a custom footer view to display at the bottom of the preview screen.

viewController	The ECG preview view controller.
return	A custom footer view, or nil to use the default footer.

```
- (UIView *
_Nullable)headerViewForEcgPreviewViewController:(ACEcgPreviewViewControlle
r * _Nonnull)viewController;
```

Asks the delegate for a custom header view to display at the top of the preview screen.

viewController	The ECG preview view controller.
----------------	----------------------------------

return	A custom header view, or nil to use the default header.
--------	---

```
- (UIBarButtonItem *  
_Nullable)rightItemViewForEcgPreviewViewController:(ACEcgPreviewViewContro  
ller * _Nonnull)viewController;
```

Ask the delegate for a custom right navigation bar button item.

viewController	The ECG preview view controller.
----------------	----------------------------------

return	A custom right bar button item, or nil to use the default item.
--------	---

```
- (UIBarButtonItem *  
_Nullable)leftItemViewForEcgPreviewViewController:(ACEcgPreviewViewControl  
ler * _Nonnull)viewController;
```

Ask the delegate for a custom left navigation bar button item.

viewController	The ECG preview view controller.
----------------	----------------------------------

return	A custom left bar button item, or nil to use the default item.
--------	--

```
-  
(BOOL)supportLandscapeModeForEcgPreviewViewController:(ACEcgPreviewViewCon  
troller * _Nonnull)viewController;
```

Asks the delegate whether the ECG preview screen should support landscape orientation.

viewController	The ECG preview view controller.
----------------	----------------------------------

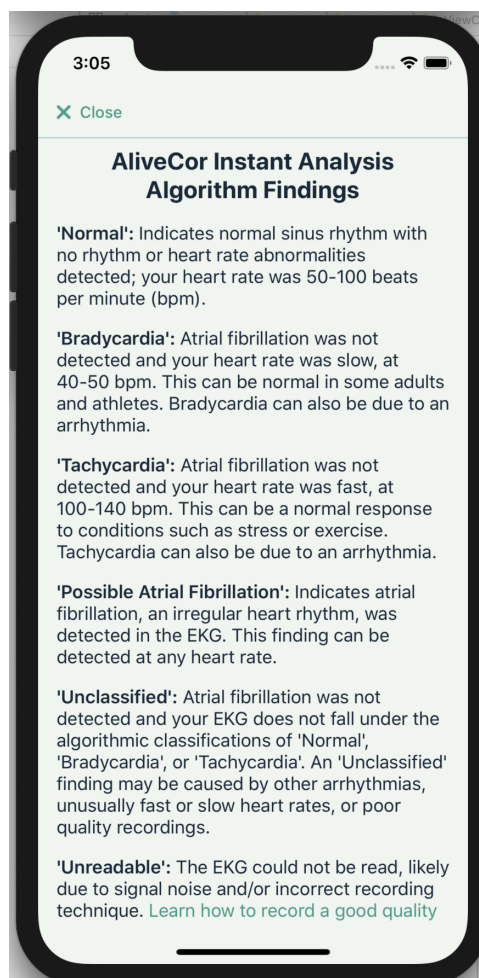
return	YES to allow landscape mode; NO otherwise.
--------	--

```
- (void)ecgPreviewViewController:(ACEcgPreviewViewController *
```

<code>_Nonnull)viewController</code> <code>didInvertRecord:(ACEcgRecord * _Nullable)record;</code>	
Notifies the delegate that the user tapped the Invert button.	
viewController	The ECG preview view controller.
record	The ECG record after inversion.

ACKAlgorithmDeterminationsViewController

The controller gives access to the list of determinations supported by the Kardia application.



Methods

<code>init(algorithmPackage: String?)</code>	
Creates a new determinations view controller.	
<code>algorithmPackage</code>	The algorithm package identifier. Pass <code>"kaiv2"</code> (case-insensitive) to enable Kardia AI v2 determinations; all other values fall back to v1.

ACKEcgFileView

A view class for displaying an ECG chart.



Properties

```
@property(nonatomic) UIColor *traceColor;
```

The color of the ECG's chart.

```
@property(nonatomic) UIColor *ecgBackgroundColor;
```

The color of the ECG's background grid.

```
@property(nonatomic, assign) BOOL antialiasing;
```

Defines whether antialiasing is enabled or not.

```
@property(nonatomic, readonly, assign) NSInteger xScale;
```

The X axis scale in percentage.

```
@property(nonatomic, readonly, assign) NSInteger yScale;
```

The Y axis scale in percentage.

```
@property(nonatomic, readonly, assign) CGFloat majorGridPixelSize;
```

The size of the grid in pixels.

```
@property(nonatomic, readonly, assign) NSInteger pixelLength;
```

The size of the view in pixels.

```
@property(nonatomic, readonly, assign) BOOL inverted;
```

Defines whether the ECG chart is inverted or not.

```
@property(nonatomic, weak) id <ACEcgFileViewDelegate> delegate;
```

The delegate for tracking touches in ACEcgFileView.

```
@property(nonatomic) ECGDisplayMode maximumDisplayMode;
```

Sets the most leads that should be displayed.

```
@property(nonatomic, readonly) ECGDisplayMode displayMode;
```

Actual number of leads being displayed.

```
@property(nonatomic) NSString *filename;
```

Optional filename associated with the rendered ECG content.

Methods

```
- (void)renderEcg:(ACEcgRecord *)ecg
    xScale:(NSInteger)xScale
    yScale:(NSInteger)yScale
scrollingDisabled:(BOOL)scrollingDisabled;
```

Renders ECG.

ecg	The ecg record was made using one of AliveCor's supported devices.
xScale	The X axis scale in percents.
yScale	The Y axis scale in percents.
scrollingDisabled	Defines whether scrolling for the ECG view will be enabled or disabled.

```
- (void)renderEcgAtPath:(NSString *)ecgPath
    filterType:(ACKFilterType)filterType
    isInverted:(BOOL)isInverted
    xScale:(NSInteger)xScale
    yScale:(NSInteger)yScale
```

scrollingDisabled:(BOOL)scrollingDisabled;	
Renders the ECG from the ACKECgRecord instance.	
ecgPath	The path to the ECG record.
filterType	The filter type that was used to record ECG.
isInverted	Whether ECG is inverted or not.
xScale	The X axis scale in percentages.
yScale	The Y axis scale in percentages.
scrollingDisabled	Defines whether scrolling for the ECG view will be enabled or disabled.

- (void)invertECG;	
Inverts the ECG chart.	

- (void)reset;	
Resets the ECG chart.	

- (void)setXScale:(NSInteger)percent;	
Setting X axis scale in percentage.	

- (void)setYScale:(NSInteger)percent;	
Setting Y axis scale in percentage.	

ACKScrollView

A scrollable container for displaying ECG charts.

`ACKScrollView` embeds an `ACKEcgFileView` inside a `UIScrollView`, providing horizontal scrolling for long ECG traces. It is typically used to present recorded ECG data that extends beyond the visible screen width.



Properties

`@property (nonatomic, readonly) UIScrollView *scrollView;`

The underlying scroll view that enables horizontal scrolling.

`@property (nonatomic) ACKEcgFileView *fileView;`

The ECG file view being displayed inside the scroll view.

`@property (nonatomic) BOOL scrollEnabled;`

Indicates whether scrolling is enabled.

Properties

definition

Methods

`- (instancetype)initWithEcgFileView:(ACKEcgFileView *)fileView;`

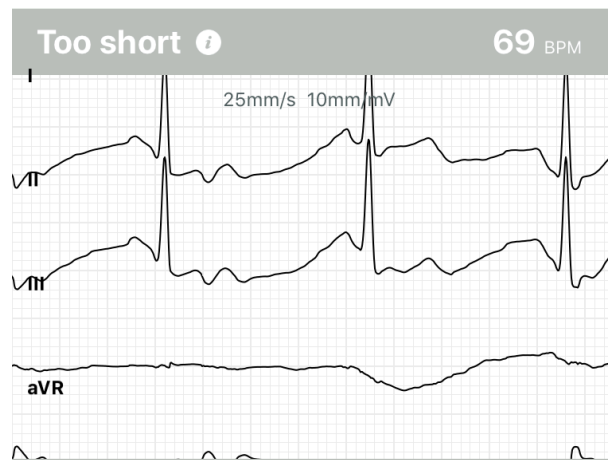
Initializes a scrollable ECG view with the given file view.

fileView	The `ACKEcgFileView` instance to display.
return	An initialized scroll view instance.

ACKRecordingResultTraceView

An ECG recording view that displays the waveform along with lead labels, scaling factors, Kardia AI determination, and average heart rate.

Using this class requires adopting `ACKRecordingResultTraceViewDelegate`.



Properties

```
@property (nonatomic, readonly) UIButton *invertButton;
```

The button used to invert the ECG trace.

Methods

```
- (instancetype)initWithEcg:(ACEcgRecord *)ecg
    fileView:(ACEcgFileView *)fileView
    largeLayout:(BOOL)largeLayout
    supportsLandscape:(BOOL)supportsLandscape
    delegate:(id<ACKRecordingResultTraceViewDelegate>)delegate;
```

Initializes a trace view with a given ECG, file view, layout mode, and delegate.

ecg	The ECG record to display.
fileView	The file view responsible for rendering the waveform.
largeLayout	Pass YES to use the large layout; NO for compact.

supportsLandscape	Pass YES to allow landscape layout; NO otherwise.
delegate	The delegate that receives interaction callbacks.
return	An initialized trace view.

- (NSInteger)standardHeightForSingleLead;	
Returns the standardized height for single-lead display, given the current recording mode, layout, and orientation.	
return	The recommended height in points.

ACKRecordingResultTraceViewDelegate

Delegate methods for customizing and responding to interactions in `ACKRecordingResultTraceView`.

Methods

- (CGFloat)recordingResultTraceViewHeightForLeadsConfig:(ACKLeadsConfig)leadsConfig;	
Returns a custom height for the trace view based on the current leads configuration. If not implemented, the view falls back to Auto Layout constraints.	
leadsConfig	The active leads configuration.
return	The desired height in points.

- (void)recordingResultTraceViewPressedInvertEcgButton;	
Notifies the delegate that the Invert ECG button was pressed.	

- (void)recordingResultTraceViewPressedAlgorithmInfoButton;	
---	--

Notifies the delegate that the Algorithm Info button was pressed.

```
- (void)recordingResultTraceViewPressedAlgorithmInfoButton;
```

Notifies the delegate that the Algorithm Info button was pressed.

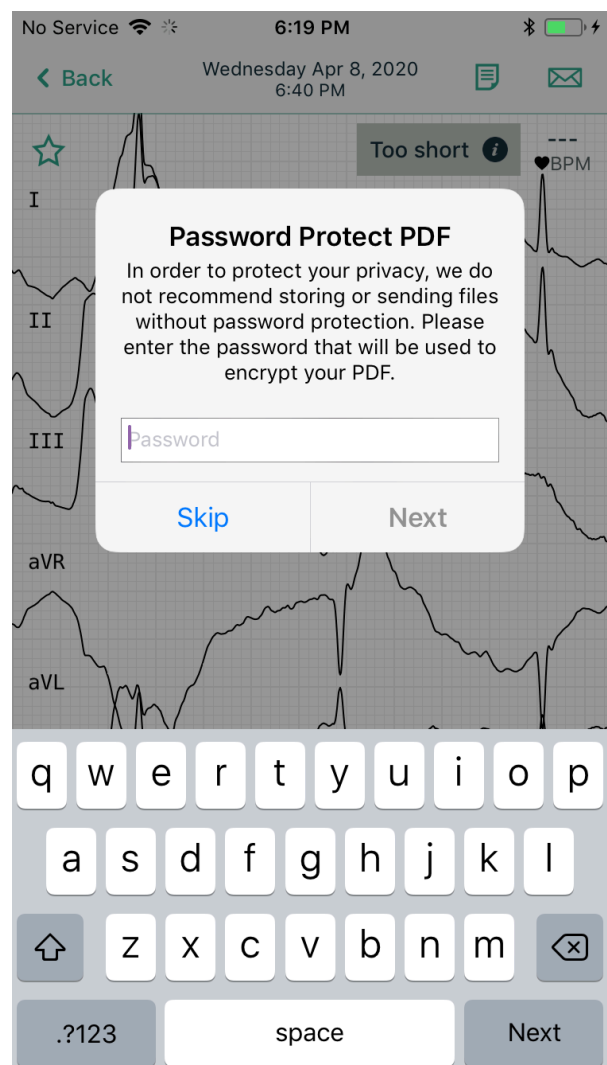
ACKPDFInteractionController

A controller that manages the creation and presentation of ECG PDF reports.

`ACKPDFInteractionController` takes an instance of `ACKPDFConfig` that defines what data should be included in the report. When

`presentPdfPreviewForConfig:onViewController:supportsLandscape:showEncryptionPrompt:encryptionRequired:` is called, the SDK generates the PDF file, optionally requests encryption, and presents a preview to the user.

- Note: Using this class requires adopting the `ACKPDFInteractionControllerDelegate` protocol. - Important: The SDK does ****not**** perform encryption itself. The host application is responsible for implementing encryption logic in compliance with regional requirements.



Properties

```
@property (nonatomic, weak, nullable) id<ACKPDFInteractionControllerDelegate> delegate;
```

The delegate that receives PDF creation and encryption events.

Methods

```
- (instancetype)initWithConfig:(ACKPDFConfig *)config  
                           error:(ACKError * _Nullable *_Nullable)error;
```

Initializes a PDF interaction controller with the given configuration.

config	The configuration of the PDF report.
error	An optional error pointer set if initialization fails.
return	A new instance of the PDF interaction controller.

```
- (instancetype)initWithConfig:(ACKPDFConfig *)config;
```

Initializes a PDF interaction controller with the given configuration.

config	The configuration of the PDF report.
return	A new instance of the PDF interaction controller.

```
- (void)presentPdfPreviewForConfig:(ACKPDFConfig *)config  
                                onViewController:(__kindof UIViewController *)viewController  
                                supportsLandscape:(BOOL)supportsLandscape  
                                showEncryptionPrompt:(BOOL)showEncryptionPrompt  
                                encryptionRequired:(BOOL)encryptionRequired;
```

Presents the PDF preview for the specified configuration.

config	The configuration of the PDF report.
viewController	The parent view controller on which the preview should be presented.
supportsLandscape	Indicates whether landscape orientation is supported.

showEncryptionPrompt	Whether to display an encryption prompt during PDF creation.
encryptionRequired	Indicates whether encryption is mandatory (for example, in certain countries such as Norway).

ACKPDFInteractionControllerDelegate

Protocol defining methods for monitoring and managing PDF generation events.

Methods

<pre>- (void)pdfInteractionController:(ACKPDFInteractionController * _Nonnull)controller willPresentPdfForConfig:(ACKPDFConfig *)config;</pre>	
Notifies the delegate that PDF generation is about to begin.	
controller	The PDF interaction controller.
config	The configuration defining the content of the PDF report.

<pre>- (void)pdfInteractionController:(ACKPDFInteractionController * _Nonnull)controller didPresentPdf:(NSString * _Nullable)pdfPath forConfig:(ACKPDFConfig *)config withError:(ACKError * _Nullable)error;</pre>	
Notifies the delegate that PDF generation has completed.	
controller	The PDF interaction controller.
config	The configuration of the PDF report.
pdfPath	The file path of the generated PDF report, or `nil` if generation failed.
error	The error that occurred during generation, or `nil` if successful.

<pre>- (void)pdfInteractionController:(ACKPDFInteractionController *</pre>	
--	--

```
_Nonnull)controller
    encryptPDFWithPassword:(NSString * _Nullable)password
        withConfig:(ACKPDFConfig *)config
        completionHandler:(void (^)(NSError * _Nullable error,
NSString * _Nullable pdfPath))completionHandler;
```

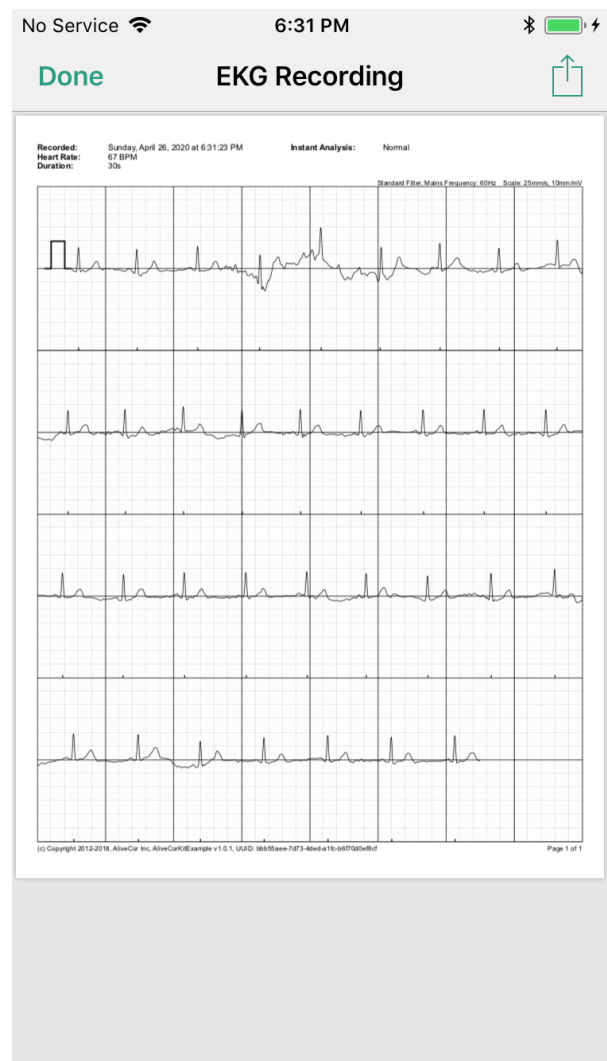
Requests the delegate to handle PDF encryption.

controller	The PDF interaction controller.
password	The password to be used for PDF encryption.
config	The configuration of the PDF report.
completionHandler	A block to be executed upon completion of the encryption process. The handler returns either the file path of the encrypted PDF or an error if encryption fails.

ACKPDPPreviewController

A view controller for displaying a PDF file.

`ACKPDPPreviewController` presents a PDF file using Quick Look (`QLPreviewController`). The file is expected to be temporary and will be automatically deleted when the view controller is dismissed. This class is typically used to display generated ECG reports or other transient PDF documents within the AliveCor SDK.



Methods

```
+ (nullable instancetype)viewControllerWithPdfFilePath:(NSString *)filePath
                        navigationBarTitle:(NSString *)title
                        supportsLandscape:(BOOL)supportsLandscape;
```

Creates and returns a PDF preview controller configured to display the specified file. The PDF file is expected to be a temporary file that will be deleted automatically after the controller is dismissed.

filePath	The full path to the PDF file.
----------	--------------------------------

title	The title displayed in the navigation bar.
supportsLandscape	Indicates whether landscape orientation is supported.
return	A configured instance of `ACKPDPPreviewController`, or `nil` if initialization fails.

<pre>+ (nullable instancetype)viewControllerWithPdfFilePath:(NSString *)filePath navigationBarTitle:(NSString *)title;</pre>	
Creates and returns a PDF preview controller configured to display the specified file. The PDF file is expected to be a temporary file that will be deleted automatically after the controller is dismissed.	
filePath	The full path to the PDF file.
title	The title displayed in the navigation bar.
return	A configured instance of `ACKPDPPreviewController`, or `nil` if initialization fails.

ACKPDFConfig

A configuration object that defines what data should appear in a generated PDF report and how that data is presented.

`ACKPDFConfig` encapsulates all the necessary information for building a PDF report, including the ECG record, patient metadata, filter settings, and display preferences.

Properties

```
@property (nonatomic, copy, nullable) ACKECgRecord *ecgRecord;
```

The ECG record to be displayed in the PDF report.

```
@property (nonatomic, copy, nullable) ACKPDFMetadata *metadata;
```

The metadata associated with the ECG record, such as patient and measurement details.

```
@property (nonatomic) ACKFilterType filterType;
```


The filter type applied when rendering the ECG in the PDF.

```
@property (nonatomic, copy, nullable, readonly) ACKPDFDisplaySettings *displaySettings;
```

The display settings that control layout, grid style, and other rendering options.

Methods

```
- (instancetype)initWithEcg:(ACEcgRecord *)ecgRecord;
```

Creates a new PDF configuration for the specified ECG record.

ecgRecord	The ECG record for which the PDF report should be generated.
-----------	--

return	A fully initialized `ACKPDFConfig` instance.
--------	--

ACKPDFDisplaySettings

This class contains various display settings to generate PDF reports.

Properties

```
@property (nonatomic) ACKFilterType filterType;
```

The filter type that should be applied for the ECG record. By default, filterType is equal to ACKFilterTypeEnhanced.

```
@property (nonatomic, copy) ACKPaperSize paperSize;
```

The paper size that would be applied for the PDF report. By default, paperSize is equal to ACKPaperSizeA4.

```
@property (nonatomic) NSInteger xScale;
```

The xScale for the pdf report. By default, xScale = 100

```
@property (nonatomic) NSInteger yScale;
```

The yScale for the pdf report. By default yScale = 100

```
@property (nonatomic, copy, nullable) NSString *logoName;
```

logo on each page including the cover page.

```
@property (nonatomic, copy, nullable) NSString *coverPageFooterLogoName;
```

footer logo on the cover page.

`@property (nonatomic, copy, nullable) NSString *fileName;`

The filename that would be assigned to the PDF report. By default, the name is equal to "ECG-<uuid>.pdf".

`@property (nonatomic) BOOL isFooterWithRecordingInfo;`

If YES, the footer of the PDF will contain the recording info: Recorded software and unique identifier of the ECG.

`@property (nonatomic, nullable) ACKLanguageType languageType;`

Displaying the PDF in a specific language, independent of the system's language settings.

ACKPDFMetadata

A model object containing patient information for inclusion in PDF reports.

`ACKPDFMetadata` encapsulates demographic and identifying details associated with an ECG recording. The metadata is displayed in the PDF report header or cover page and may include patient name, age, notes, and other optional data.

Properties

`@property (nonatomic, copy, nullable) NSString *patientFirstName;`

The patient's first name.

`@property (nonatomic, copy, nullable) NSString *patientLastName;`

The patient's last name.

`@property (nonatomic, copy, nullable) NSString *patientInfo;`

Additional patient information displayed on the PDF cover page below the name (maximum of two lines).

`@property (nonatomic, copy, nullable) NSString *patientDateOfBirth;`

The patient's date of birth.

`@property (nonatomic, copy, nullable) NSString *patientAge;`

The patient's age.

```
@property (nonatomic, copy, nullable) NSString *notes;
```

Notes associated with the patient or ECG recording.

```
@property (nonatomic, copy, nullable) NSString *tags;
```

Tags or identifiers related to the patient or ECG record.

```
@property (nonatomic, copy, nullable) NSNumber *male;
```

The patient's sex. A Boolean value where `YES` indicates male.

```
@property (nonatomic, copy, nullable) NSNumber *recordedByUser;
```

Indicates whether the ECG was recorded by a registered user (`YES`) or a guest (`NO`).

Methods

```
- (BOOL)isEqualToACKPDFMetadata:(ACKPDFMetadata *)metadata;
```

Compare two `ACKPDFMetadata` objects for equality.

metadata	Another metadata object to compare.
return	`YES` if both metadata objects contain the same values; otherwise, `NO`.

ACKPDFReport

A utility class for exporting ECG recordings to PDF format.

`ACKPDFReport` provides a set of class methods for generating ECG PDF reports. Each method accepts configuration parameters that define the layout, filter type, metadata, and Kardia AI evaluation results to include in the exported document.

The generated PDF is written to a temporary file, and the method returns the file path of the resulting report.

Methods

```
+ (nullable NSString *)pdfPathForConfig:(ACKPDFConfig *)config
                                error:(NSError * _Nullable * _Nullable)error;
```

Generates a PDF report using the specified configuration.

config	The PDF report configuration.
error	If an error occurs, upon return contains an `NSError` object describing the problem.
return	The file path to the generated PDF report, or `nil` if generation failed.

```
+ (nullable NSString *)pdfPathForConfig:(ACKPDFConfig *)config;
```

Generates a PDF report using the specified configuration.

config	The PDF report configuration.
return	The file path to the generated PDF report, or `nil` if generation failed.

```
+ (nullable NSString *)pdfFromTheAtcFilePath:(NSString *)atcFilePath
                                uuid:(NSString *)uuid
    determinationResult:(nullable NSString *)determinationResult
                allowPVC:(BOOL)allowPVC
    averageHeartRate:(CGFloat)averageHeartRate
        recordedAt:(nullable NSString *)recordedAt
        isInverted:(BOOL)isInverted
        metadata:(nullable ACKPDFMetadata *)metadata
displaySettings:(nullable ACKPDFDisplaySettings *)settings
    algorithmPackage:(nullable NSString *)algorithmPackage
    averageBeats:(nullable NSArray *)averageBeats
        beats:(nullable NSArray *)beats
        intervals:(nullable NSArray *)intervals
                                error:(NSError * _Nullable * _Nullable)error;
```

Generates a PDF report directly from the provided ECG data and parameters.

atcFilePath	The file path to the `.atc` ECG data file.
uuid	The unique identifier displayed in the PDF footer.

determinationResult	The Kardia AI determination result to be displayed in the title.
allowPVC	Indicates whether PVC (Premature Ventricular Contractions) data should be drawn.
averageHeartRate	The average heart rate (bpm) to display in the PDF.
recordedAt	The recording timestamp to display in the PDF.
isInverted	If `YES`, renders the ECG trace inverted.
metadata	The patient metadata information.
settings	The display settings for rendering the PDF.
algorithmPackage	The algorithm package identifier (e.g., `"kaiv2"` for Kardia AI v2 support).
averageBeats	The average beat waveform data (used for AI v2).
beats	The list of detected beats to display.
intervals	The R–R interval data (used for AI v2).
error	If an error occurs, upon return contains an `ACKError` object describing the problem.
return	The file path to the generated PDF report, or `nil` if generation failed.

ACKPDFErrorCode

The error class. Defines error type that occurred while creating a PDF report.

Error types	
ACKPDFErrorCodeEcgPathNotFound	The ECG path was not found in the associated `ACEcgRecord`.
ACKPDFErrorCodeEcgMissing	An ECG recording is required to generate the PDF.
ACKPDFErrorCodeConfiguration	A mandatory configuration property is missing.
ACKPDFErrorCodeFileOpen	The ECG file could not be opened.

ACKPDFErrorCodeEmptySamples

The ECG file contains no samples.

ACKWebLinks

A configuration class that defines customizable help links used across various HUD views.

`ACKWebLinks` provides a centralized way to override default web links shown in help dialogs and error HUDs throughout the SDK. Each property corresponds to a specific error or instruction context. If not customized, the default AliveCor help URLs are used.

Properties

@property (nonatomic, copy, nullable) NSString *bluetoothNotAuthorized;

Help link displayed when Bluetooth authorization fails.

Default: `https://www.alivecor.com/app-redirect/i-need-help-pairing-error-bt-not-authorized`

@property (nonatomic, copy, nullable) NSString *bluetoothError;

Help link displayed for generic Bluetooth connection errors.

Default: `https://www.alivecor.com/app-redirect/i-need-help-bluetooth-error`

@property (nonatomic, copy, nullable) NSString *microphoneAccess;

Help link displayed when microphone access is denied.

Default: `https://www.alivecor.com/app-redirect/i-need-help-mic-error-ios`

@property (nonatomic, copy, nullable) NSString *kardiaMobileSingleLead;

Help link displayed for Kardia Mobile single-lead recording instructions.

Default: `https://www.alivecor.com/app-redirect/i-need-help-recording-km`

@property (nonatomic, copy, nullable) NSString *electricalInterference;

Help link displayed for electrical interference errors.

Default: `https://www.alivecor.com/app-redirect/i-need-help-electrical-interference`

@property (nonatomic, copy, nullable) NSString *tooShort;

Help link displayed when an ECG recording is too short (between 10 and 30 seconds).

Default: `https://www.alivecor.com/app-redirect/i-need-help-too-short`

@property (nonatomic, copy, nullable) NSString *triangleNotFound;

Help link displayed when the Triangle device's signal cannot be detected.

Default:

`https://www.alivecor.com/app-redirect/i-need-help-pre-recording-error-device-not-found`

`@property (nonatomic, copy, nullable) NSString *replaceBattery;`

Help link displayed when the device battery is critically low.

Default: `https://www.alivecor.com/app-redirect/i-need-help-battery-critical`

`@property (nonatomic, copy, nullable) NSString *triangleSixLead;`

Help link displayed for six-lead (6L) recording instructions.

Default: `https://www.alivecor.com/app-redirect/i-need-help-recording-6l-six-lead`

`@property (nonatomic, copy, nullable) NSString *triangleSingleLead;`

Help link displayed for single-lead recording instructions (6L device in single-lead mode).

Default: `https://www.alivecor.com/app-redirect/i-need-help-recording-6l-single-lead`

`@property (nonatomic, copy, nullable) NSString *unreadable;`

Help link displayed for unreadable ECG recordings due to excessive noise or artifacts.

Default: `https://www.alivecor.com/app-redirect/i-need-help-unreadable`

ACKBluetoothPairingController

A controller that manages the Bluetooth pairing process for devices supported by the SDK.

`ACKBluetoothPairingController` encapsulates all logic required for pairing Bluetooth-based AliveCor devices, including retry handling and background/foreground transitions.

Typically, Bluetooth pairing is managed automatically by `ACEcgMonitorViewController`. You should use this class and its delegate only if you need to implement a fully custom Bluetooth pairing user experience.

Properties

`@property (nonatomic, nullable, copy, readonly) ACKDeviceType deviceType;`

The type of device being paired.

Methods

<pre>- (instancetype)initWithDeviceType:(ACKDeviceType)deviceType delegate:(id<ACKBluetoothPairingControllerDelegate>)delegate;</pre>	
Creates and initializes a Bluetooth pairing controller for the specified device type.	
deviceType	The type of device the user intends to pair.
delegate	The delegate that receives pairing state updates and events.
return	A configured instance of `ACKBluetoothPairingController`.

ACKBluetoothPairingControllerDelegate

Defines the delegate methods for monitoring and responding to Bluetooth pairing events.

The `ACKBluetoothPairingControllerDelegate` protocol provides callbacks that inform the delegate about Bluetooth availability, device discovery, successful pairing, and connection errors during the pairing process.

Methods

<pre>- (void)bluetoothControllerReadyToScan:(ACKBluetoothPairingController *)bluetoothDeviceController;</pre>	
Called when the Bluetooth controller is powered on and ready to scan for devices.	
bluetoothDeviceC ontroller	The Bluetooth pairing controller instance.

<pre>- (void)bluetoothDeviceController:(ACKBluetoothPairingController *)bluetoothDeviceController didDiscoverTriangleDevice:(ACKDevice *)device;</pre>	
Called when a supported ECG recording device (e.g., Kardia device) is discovered during scanning.	
bluetoothDeviceC ontroller	The Bluetooth pairing controller instance.
device	The discovered device.


```
-  
(void)bluetoothDeviceControllerPairingDidSucceed:(ACKBluetoothPairingController *)bluetoothDeviceController;
```

Called when a device is successfully paired.

bluetoothDeviceController	The Bluetooth pairing controller instance.
---------------------------	--

```
-  
(void)bluetoothDeviceControllerDeviceNotFound:(ACKBluetoothPairingController *)bluetoothDeviceController;
```

Called when no device could be found during the pairing process.

bluetoothDeviceController	The Bluetooth pairing controller instance.
---------------------------	--

```
- (void)bluetoothDeviceController:(ACKBluetoothPairingController *)bluetoothDeviceController  
    didEncounterConnectionError:(ACKError *)error;
```

Called when an error occurs during the device pairing or connection process.

bluetoothDeviceController	The Bluetooth pairing controller instance.
---------------------------	--

error	The error that occurred.
-------	--------------------------

```
- (void)didPairDevice:(ACKDevice *)device;
```

Called when an ECG recording device has been successfully connected.

device	The connected device.
--------	-----------------------

ACKStreamingFilter

A utility class for applying real-time digital filters to ECG samples.

`ACKStreamingFilter` provides real-time filtering of raw ECG signal samples using one of the supported AliveCor filter types.

This class is designed for cases where the app has access to raw ECG data (e.g., from a live data stream or stored file). It does **not** provide access to the same filtered data displayed on the standard ECG recording screen (`ACEcgMonitorViewController`).

Typical usage:

1. Establish access to a raw ECG data source (file, stream, etc.).
2. Create an instance of `ACKStreamingFilter` with the desired configuration.
3. Pass each incoming sample through `filterSample:` to obtain a filtered output.

Methods

```
- (nullableinstancetype)initWithFilterType:(ACKFilterType)filterType
                                sampleRate:(ACKSampleRate)sampleRate
                                mainsFrequency:(ACKMainsFrequency)mainsFrequency
                                error:(NSError * _Nullable *)error;
```

Creates an instance of `ACKStreamingFilter` with the specified configuration.

filterType	The filter type to apply to the ECG signal.
sampleRate	The sampling frequency of the ECG signal.
mainsFrequency	The power-line frequency where the ECG was recorded (`ACKMainsFrequency50Hz` or `ACKMainsFrequency60Hz`).
error	If an error occurs during initialization, this parameter is set to an `NSError` object describing the issue.
return	A new instance of `ACKStreamingFilter`, or `nil` if initialization fails.

```
- (double)filterSample:(double)sample;
```

Filters a single ECG sample using the configured filter.

sample	The raw ECG sample value (in millivolts).
return	The filtered ECG sample.

```

NSArray<NSString *> *rawDataArray = [self rawEcgData]; // "time(seconds),signal(mV)"
               "0.003333333,-0.161590576"
NSError *error;
ACKStreamingFilter *filter = [[ACKStreamingFilter alloc] initWithFilterType:ACKFilterTypeOriginal
                               sampleRate:ACKSampleRate300Hz mainsFrequency:ACKMainsFrequency60Hz error:&error];

if (error) {
    NSLog(@"ACKStreamingFilter failed with error: %@", error.localizedDescription);
} else {
    //Print non filtered and filtered values from the array
    for (NSString *pair in rawDataArray) {
        NSArray *item = [pair componentsSeparatedByString:@" "];
        double mvValue = [item[1] doubleValue];
        double filteredValue = [filter filterSample: mvValue];
        NSLog(@"originalValue: %f, filteredValue: %f", mvValue, filteredValue);
    }
}

```